



GROUP 2 – PAYMENT ENROLMENT SYSTEM

ABSTRACT

Written by Peter Maravgakis on December 21st.

GROUP 2:

CHRYSOSTOMOS DEMOSTHENOUS, ANDREAS KARPASITIS,

PETER MARAVGAKIS, IRENEOS PAVLOU, CONSTANTINOS STEPHANIDES

This paper covers the development of the Payment Enrolment System by Group 2 over the duration of the courses Software Engineering I(CSC411/CSE325/CSC650) and Software Engineering II(CSE410/CSC412) taken between Spring 2025 and Fall 2025-6.



TERMS, ACRONYMS, AND ABBREVIATIONS

- PES: Payment Enrollment System
- GDPR: General Data Protection Regulation.
- HR: Human Resources
- UC: Use Case
- PSC: Pseudocode
- AC: Actor-Centric Use Case
- DFD: Data Flow Diagram
- CFD: Control Flow Diagram

TABLE OF CONTENTS

ABSTRACT.....	1
Terms, Acronyms, and Abbreviations	1
Table of Contents.....	1
PROJECT EXECUTIVE SUMMARY	1
Table of Contents.....	2
List of Figures	2
List of Tables	2
Terms, Acronyms, and Abbreviations	2
Introduction	2
Project Management Plan	2
Risk Analysis	2
Gantt Chart – Deliverables and Milestones.....	2
Monitoring and Controlling Mechanisms	3
Project Documentation & Collaboration	3
Communication & Reporting.....	3
Risk Management & Issue Tracking	3
Requirements and Specifications	3
Stakeholders of the System.....	3
Functional and Non-Functional Requirements.....	3
System Architecture.....	4
High-Level Depiction of PES	4
UML Component Diagram.....	4
UML Deployment Diagram	4
System Design	4
Use Cases.....	4
Related Diagrams.....	4
Data Flow Diagram (DFD)	4
UML Class Diagram.....	4
Sequence Diagram.....	4
Control Flow Diagram	4
Test Plan	4
Test Cases	4
Test case 1 (TC01):.....	4
Test Case 2 (TC02):	4
Test Case 3 (TC03):	4
Test Case 4 (TC04): Leave Request and Approval System	4
Test Case 5 (TC05): Email Notification System	4
Traceability of Requirements with Use Cases and Test Cases	4

Professional Standards	5
Mapping Table.....	5
PROJECT PROGRESS REPORT	6
Functional Requirements	7
System Design	8
Progress report document CHANGELOG	9
GANTT CHART (accessible here)	10
System module pseudocode.....	10
FR-00A: MAIN ROUTING SYSTEM	10
FR-01: MAIN LOGIN FUNCTION	10
FR-02: EMPLOYEE ONBOARDING Function:.....	10
FR-03: RECORD ATTENDANCE Function:	10
FR-04: LEAVE REQUEST FUNCTION:	10
FR-05: LEAVE MANAGE Function:.....	10
FR-06: PAYROLL PROCESSING FUNCTION:	10
FR-07: PAYMENT DISBURSEMENT FUNCTION:	10
FR-08: PAYSIP ACCESS FUNCTION:	10
FR-09: FINANCIAL REPORT GENERATION FUNCTION:	10
FR-10: SALARY UPDATE MODULE:	10
FR-11: USER AUTHENTICATION MODULE:	10
FR-12: PORTAL MODULES:.....	10
TEST CASES	10
TC-01: USER LOGIN AUTHENTICATION (FR-01, FR-11, FR-12).....	10
TC-02: USER LOGIN WITH MISTYPED CORRECT DATA (FR-01, FR-11, FR-12)	11
TC-03: USER LOGIN ATTEMPTED DESTRUCTIVE BYPASS (FR-01, FR-11, FR-12)	12
TC-04: USER LEAVE REQUEST FUNCTION WITH CORRECT INFORMATION (FR-04).....	13
TC-05: USER LEAVE REQUEST FUNCTION WITH INVALID INFORMATION (FR-04).....	14
TC-07A: EMPLOYEE LEAVE MANAGEMENT ACCEPT (FR-05, FR-04).....	15
TC-07B: EMPLOYEE LEAVE MANAGEMENT REJECT (FR-05, FR-04)	16
TC-08A: EMPLOYEE ONBOARDING ADD (FR-02)	17
TC-08B: EMPLOYEE ONBOARDING REMOVE (FR-02)	18
TC-12E: EMPLOYEE PORTAL ACCESS (FR-12, FR-01, FR-00A)	19
TC-12H: HR PORTAL ACCESS (FR-12, FR-01, FR-00A)	20
TC-12F: FINANCE PORTAL ACCESS (FR-12, FR-01, FR-00A)	21
TC-12A: ADMIN PORTAL ACCESS (FR-12, FR-01, FR-00A).....	22
TC-13: VIEWING AND downloading pdf (FR-08)	23
TC-14: EMPLOYEE ONBOARDING/OFFBOARDING (FR-02).....	24
TC-15: Update employee personal data(FR-02b).....	25
TC-16: PORTALS – EMPLOYEE portal (FR-12).....	26
TC-17: PORTALS – HR PORTAL (FR-12)	26

USER MANUAL26

THE LOGIN PAGE26

THE EMPLOYEE PORTAL26

THE HR PORTAL26

THE FINANCE PORTAL26

SUBMODULES26

VIEW PAYSLEIPS26

REQUEST PERSONAL LEAVE26

MANAGE LEAVE REQUESTS26

EMPLOYEE MANAGEMENT.....26

UPDATE EMPLOYEE DETAILS26

SEMESTER 1 (SPRING 2025)

PROJECT EXECUTIVE SUMMARY

Software Engineering Project Final Report

Project workload rules:

1. Each group member should be **responsible to monitor the progress** of at least one section as depicted in the table below.

Section	Leading Team Member	Team 2 LET'S GO
Executive Summary	All team members	Everyone
Terms, Acronyms, and Abbreviations	All team members	Everyone
1.Introduction	Team member 1	Peter (Green Highlight)
4.System Architecture		
7.Professional standards		
2.Project Management Plan	Team member 2	Ireneos
3.Requirements and Specifications	Team member 3	Andreas
5.System Design	Team member 4	Constantinos
6.Test Plan	Team member 5	Tommy

Note that the workload of sections 1, 4 and 7 is much less than the rest of the sections, that's why they are grouped together.

Payment Enrollment System

Executive Summary

This paper provides a high-level overview of the Payment Enrollment System (PES) project, summarizing its core objectives, scope, and deliverables. It highlights the system's purpose: to automate payroll processing, manage employee records, calculate salaries based on attendance/leave data, and generate financial reports. Key outcomes include reduced manual errors, GDPR-compliant data security, seamless banking integrations, and self-service portals for employees. This section serves as a concise snapshot for stakeholders to grasp the project's value without delving into technical details.

The following is the description from the project list file:

Objective: The Employee and Payroll Systems objective is to provide a system which manages the employee details, the Payroll activity done in a company depending upon the employees' attendance and all required calculation. The system will take care of all the payroll activities like managing each employee's attendance, the number of leaves taken by that particular employee and calculation in a very quick manner without any estimation mistakes. All the important data's will be stored in a database.

The "Employee and Payroll System" needs to maintain each employee records and calculating his/her salary depending on the workdays. The first activity is based on saving the employees details where each employee will be given a unique Employee ID. Then, based on the number of days an employee attended per month, salary will be calculated by checking the number of workdays and his/her basic salary and a salary slip will be provided/generated for reference.

Input:

- Employee details (employee no, name, address, designation, department, achievements)
- Accounts details (salary of each employee, deduction(s), income tax, social insurance, GESY, other allowance, etc.)
- Leave information (number of leave taken by each employee)

Output:

- Salary slip
- Detailed salary report
- Deduction details
- Leave information

Constraints:

- Admin must have the permission to update the employee information manually
- Manager should be able to see the details of an employee
- Salary slip, detailed salary report, deduction details, and leave and information of a particular employee must not be accessible to another employee
- The leave information must be updated automatically, if an employee takes leave.

TABLE OF CONTENTS

Table of Contents	3
List of Figures	4
List of Tables	5
Terms, Acronyms, and Abbreviations	5
1 Introduction	5
2 Project Management Plan	7
2.1 Risk Analysis	7
2.2 Gantt Chart – Deliverables and Milestones	10
2.3 Monitoring and Controlling Mechanisms	13
Project Documentation & Collaboration	14
Communication & Reporting	14
Risk Management & Issue Tracking	14
3 Requirements and Specifications	15
3.1 Stakeholders of the System	17
3.2 Functional and Non-Functional Requirements	18
4 System Architecture	20
High-Level Depiction of PES	21
UML Component Diagram	22
Deployment Diagram	23
5 System Design	24
5.1 Use Cases	24
Use Case 1: Employee Onboarding	24
Use Case 2: Attendance and Leave Management	25
Use Case 3: Payroll Processing & Salary Calculation	26
Use Case 4: Payment Disbursement	27
Use Case 5: Viewing and Downloading Pay Slips	28
Use Case 6: Compliance and Financial Reporting	29
5.2 Related Diagrams	30
5.2.1 Data Flow Diagram (DFD)	33
5.2.2 UML Class Diagram	39
5.2.3 Sequence Diagram	40

5.2.4 Control Flow Diagram 43

6 Test Plan 46

6.1 Test Cases 47

Test Case 4 (TC04): Leave Request and Approval System 50

Test Case 5 (TC05): Email Notification System 52

6.2 Traceability of Requirements with Use Cases and Test Cases 53

7 Professional Standards 53

Mapping Table 55

LIST OF FIGURES

1. Figure 1: Risk analysis table
2. Figure 2: Gantt Chart
3. Figure 3: High-level representation of the Payment Enrollment System
4. Figure 4: UML Component Diagram for PES
5. Figure 5: UML Class Diagram for PES
6. Figure 6: Deployment Diagram for PES
7. Figure 7: Employee Onboarding Sequence Diagram
8. Figure 8: Attendance & Leave Management Sequence Diagram
9. Figure 9: Payment Disbursement Control Flow Diagram
10. Figure 10: Compliance & Financial Reporting Control Flow Diagram
- 11.

LIST OF TABLES

12. Table 1: Tasks schedule breakdown
13. Table 2: Functional requirements and use cases
14. Table 3: Requirements Traceability
15. Table 4: Professional standards mapping

TERMS, ACRONYMS, AND ABBREVIATIONS

- GDPR: General Data Protection Regulation.
- PES: Payment Enrollment System
- HR: Human Resources
- UC: Use Case
- PSC: Pseudocode
- AC: Actor-Centric Use Case
- DFD: Data Flow Diagram
- CFD: Control Flow Diagram

INTRODUCTION

Managing employee payments is a fundamental yet time-consuming process in any organization. From onboarding new employees to calculating salaries and ensuring compliance with tax regulations, payroll management can become a complex administrative burden.

A **Payment Enrollment System (PES)** addresses these challenges by automating payroll processing, reducing errors, and improving efficiency within an organization's **Human Resources (HR) and Finance departments**. It will essentially be a platform for housing a list of all employees, their financial information in relation to the company (work hours per month, leave, everything in between), and the calculations of the company's salary distribution and expenses based on the payrolls of its employees.

A PES is built on a **CRUD (Create, Retrieve, Update, Delete) framework**, allowing companies to efficiently manage employee financial records. The core objectives of PES include:

- **Employee and entity onboarding:** The system enables HR teams to register new employees, contractors, and other payees in the database. Each record includes essential information such as salary structure, tax brackets, benefits, and deductions.
- **Updating financial records:** Changes such as tax rate modifications, salary increments, bonuses, or deductions for social security contributions can be updated effortlessly.
- **Automated calculations:** The system computes gross salaries, tax deductions, net salaries, and other financial adjustments based on predefined business rules and tax regulations.
- **Financial record-keeping:** All transactions, pay slips, tax deductions, and payment histories are securely stored in the system's database, ensuring **auditability and compliance** with financial laws.
- **Integration with banking and payment services:** The system facilitates direct salary transfers, reducing manual intervention and ensuring timely payments.

What does all of this tell you? This application can be easily developed using a web application!

Furthermore, by automating these critical processes, the PES significantly reduces the risk of **manual errors, compliance issues, and inefficiencies**, ultimately **enhancing workforce productivity**.

Additionally, the system offers **employee self-service features**, allowing workers to **access their payslips, tax statements, and history** through a secure web portal or mobile application, *without the help of another person*. This reduces HR's workload and increases transparency in payroll management.

The PES is essential for modern organizations seeking to optimize payroll operations, **enhance security**, and comply with regulatory standards, ultimately allowing HR teams to focus on strategic initiatives rather than administrative payroll tasks.

Section 1 End

PROJECT MANAGEMENT PLAN

Introduction by Eireneos. 

The team organized the project using a collaborative and distributed approach. Each member was assigned a major section of the documentation to lead, while still contributing feedback and edits across other parts. Responsibilities were clearly defined early in the semester to ensure accountability and efficient progress.

The work was structured around internal deadlines, aligning with key academic milestones. The team emphasized flexibility and mutual trust—avoiding micromanagement in favour of asynchronous communication. Text-based coordination on Discord, combined with shared documents on Google Docs, allowed members to update, review, and discuss content efficiently without frequent meetings.

Key tools and methods included:

1. **Google Docs:** For documentation, version tracking, and comments.
2. **Discord:** For real-time updates, queries, and informal discussion.
3. **Task Assignments:** Each major section of the report was led by one member, with clearly documented contributions.
4. **Backup & Version Control:** Regular backups and tracked edits were maintained to prevent data loss and support collaboration.

This structure enabled the team to meet milestones while adapting to challenges, laying a strong foundation for the system's development phase next semester.

RISK ANALYSIS

Risk Analysis systematically identifies potential threats to the project's success (e.g., technical failures, operational delays, or security breaches) and evaluates their probability and impact. It documents mitigation strategies to proactively address these risks, ensuring minimal disruption to timelines, functionality, or compliance. This section emphasizes preventive measures (e.g., automated alerts, backup protocols, and validation logic) and assigns ownership for risk resolution, enabling the team to navigate uncertainties confidently during development and deployment.

Section 2.1: Risk Analysis Chart

First iteration done by Peter M. 

Final iteration done by Eireneos. 

Figure 1: Risk analysis table

ID No.	Risk Type	Risk Description	Affects	Existing Control Measures	Probability Level	Impact Level	Prevention Measures
1	Operational	Manager delays leave approvals	Project	Manual follow-up via email	High	Moderate	Email reminders, dashboard alerts for pending requests
2	Technical	Scheduling conflicts (double duty)	Product	Conflict validation during schedule assignment	Moderate	High	Automated overlap detection and schedule validation
3	Human Resource	Developer unavailable during sprint	Project	Shared code access, team cross-training	Moderate	Moderate	Task reallocation and sprint flexibility
4	Data Integrity	Payroll miscalculation	Business	Manual logs of hours and salary breakdown	Low	High	Auto salary calculator with manual override feature
5	Infrastructure	System/server downtime	Product	Hosted on stable cloud platform	Low	High	Uptime monitoring, backup servers
6	Technical	Data loss or corruption	Project	Daily backups	Very Low	Very High	Weekly offline backups, rollback system
7	Process	Too many leave requests at once	Project	Leave limit enforced per week	Moderate	High	Auto-decline when minimum coverage not met
8	Usability	Staff forget login credentials	Product	Email reset feature	High	Low	Add mobile-based reset, simple username logic
9	Communication	Misunderstood requirements	Project	Weekly reviews and mockups	Moderate	Moderate	Walkthroughs and confirmation before dev begins
10	Quality Assurance	Bugs found late due to poor testing timeline	Project	Unit testing by devs	High	High	Continuous testing pipeline, QA involvement from Sprint 1
11	Security	Unauthorized access to manager panel	Product	Role-based access control	Moderate	High	Encrypt user roles and verify session on server-side
12	User Behavior	Improper use of leave request system	Project	Leave form with required fields	High	Moderate	Input validation and instructional tooltips
13	Technical	Browser incompatibility	Product	Basic cross-browser CSS rules	Moderate	Moderate	Test across Chrome, Firefox, Edge, Safari pre-deployment
14	Infrastructure	Internet outage during usage	Product	None	Moderate	High	Enable client-side caching for 7-day view
15	Compliance	Leaves not tracked correctly	Business	Manual oversight	Moderate	High	Real-time tracking dashboard for leave balance
16	Design	Non-intuitive UI affects productivity	Product	Simple layout with buttons	Low	Moderate	User testing during mockup stage
17	Scheduling	Late duty assignment	Project	Manual deadline tracking	Moderate	High	Auto-alerts for schedule publishing deadlines
18	Deployment	Delay in going live	Project	Weekly milestones	Moderate	High	Contingency buffer in final 2 weeks
19	Management	Team conflict affects deliverables	Project	Weekly check-ins	Low	Moderate	Conflict resolution plan and anonymous reporting
20	Legal	Salary dispute by personnel	Business	Basic payslip with details	Low	High	Transparent monthly logs accessible to users

◇ Risk 1: Manager Delay in Approving Leave Requests

Mitigation: Implement automated alerts and fallback logic that temporarily treats unapproved requests as “pending” without disrupting scheduling.

◇ Risk 2: Scheduling Conflicts (Double Duty Assignments)

Mitigation: Apply validation logic in the system to prevent assigning the same person to overlapping time slots or places.

◇ Risk 3: Sudden Absence of Developer During Key Phase

Mitigation: Assign a backup developer for critical components and ensure shared access to code and documentation.

◇ Risk 4: Salary Discrepancy Due to Miscalculated Leaves

Mitigation: Provide salary breakdown logs for each employee with total duty hours, leaves, and fines to avoid confusion.

◇ Risk 5: Data Loss (Due to power outage or accidental deletion)

Mitigation: Enable daily automatic database backups and include manual weekly offline backups.

◇ Risk 6: Too Many Security Personnel on Leave Simultaneously

Mitigation: Enforce a dynamic rule that auto-declines leaves if coverage for places falls below the minimum threshold.

◇ Risk 7: Requirement Misunderstanding

Mitigation: Prepare mock-ups and detailed requirement documents early and review them with stakeholders weekly.

◇ **Risk 8:** Hosting Server Downtime

Mitigation: Use a reliable cloud service with 99.9% uptime SLA and enable server health monitoring.

◇ **Risk 9:** Communication Gaps Between Team Members

Mitigation: Use centralized communication platforms with daily check-ins and transparent Trello task updates.

◇ **Risk 10:** Scope Creep

Mitigation: Lock the scope after the design phase. Any changes must go through a documented change request process.

◇ **Risk 11:** Security Personnel Forget Their Login Credentials

Mitigation: Include a “Forgot Password” feature using email/phone verification and user-friendly login system.

◇ **Risk 12:** Browser Incompatibility

Mitigation: Test UI across Chrome, Firefox, Safari, and Edge. Use responsive frameworks like Bootstrap or Tailwind CSS.

◇ **Risk 13:** Users Not Following Proper Leave Request Procedure

Mitigation: Add mandatory fields with validations and send email confirmation on submission.

◇ **Risk 14:** Manager Accidentally Approves Overlapping Duties

Mitigation: Add automatic conflict checker that flags overlapping assignments before confirmation.

◇ **Risk 15:** Internet Outage During Critical Usage

Mitigation: Cache the last 7-day schedule on the client side for temporary offline access.

◇ **Risk 16:** Unauthorized Access (Role Tampering)

Mitigation: Implement RBAC at both frontend and backend; never trust client-side role data.

◇ **Risk 17:** Payroll Module Bug Results in Salary Miscalculation

Mitigation: Include unit tests for payroll logic and display detailed salary breakdown.

◇ **Risk 18:** System Not Scaling with User Base

Mitigation: Design database and API for scalability; perform load testing.

◇ **Risk 19:** Delay in Testing Due to Late Integration

Mitigation: Use continuous integration and test modules independently.

◇ **Risk 20:** Difficulties in Updating Weekly Routines

Mitigation: Create a visual drag-and-drop scheduler with filters for availability and conflicts.

GANTT CHART – DELIVERABLES AND MILESTONES

To effectively plan our project timeline, we designed the Gantt Chart with the help of Gantt Project, a cost-free planning tool. We began by segregating our project into individual tasks and subtasks with their estimated duration, interdependence, and ownership. From this segregation, we derived major deliverables such as the completed requirements document, system design documents, various iterations of documents, and subsequent phases such as implementation and testing. Each deliverable comprises certain milestones such as "Requirements Finalized", "Design Complete", and "Tool Stack Chosen" as development milestones and quality gates.

The Gantt Chart shows graphically the project timetable, presenting how activities are sequenced and interrelated. Parallelization of some activities is done, for example, documentation tasks and tool investigation, as they can be executed independently or by various members of the team to maximize time utilization and minimize bottlenecks. The chart will help us plan and communicate effectively, especially during the second phase of the project. The chart will be a project working document in the tracking of progress, the changes in the timeline, and aligning with team goals, deadlines, and tasks.

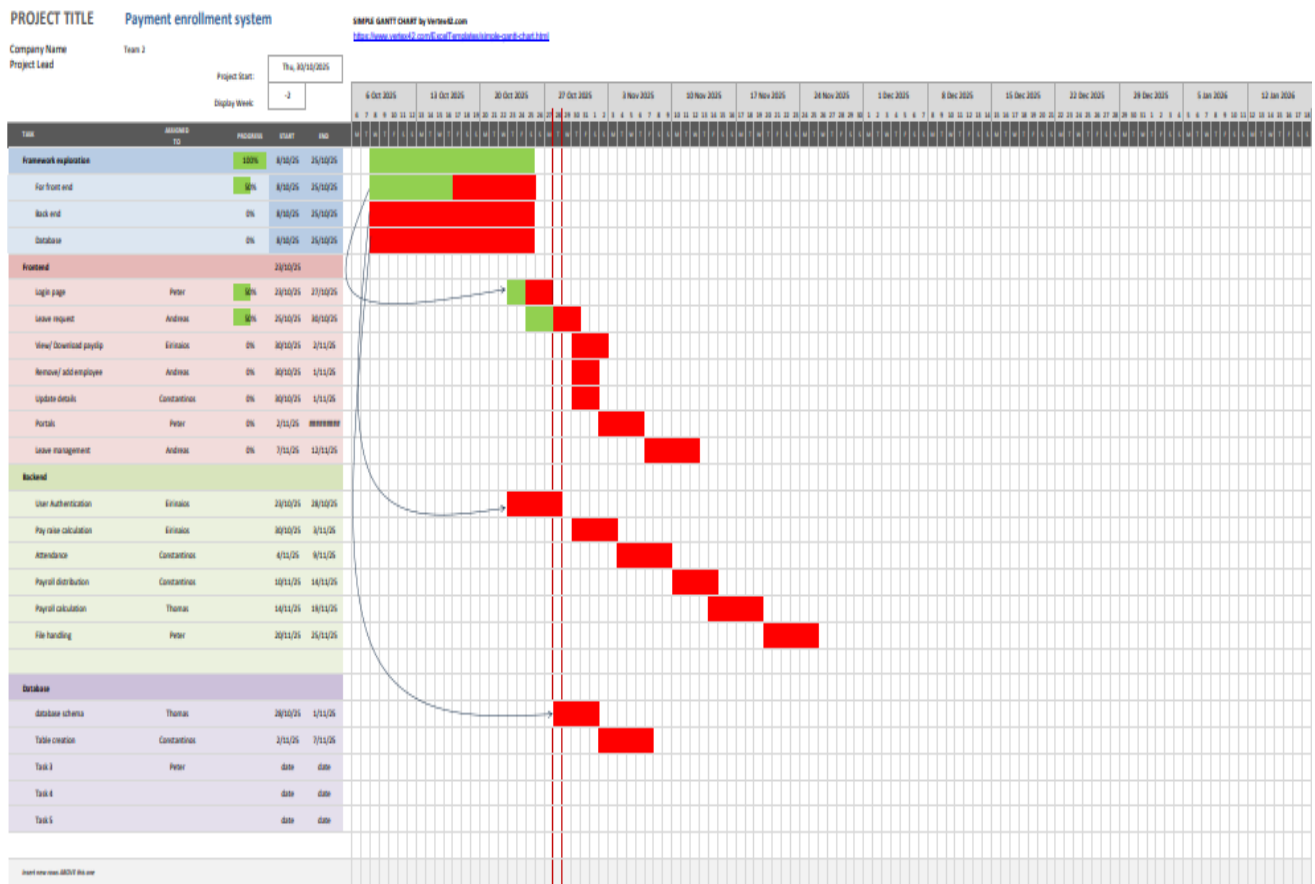
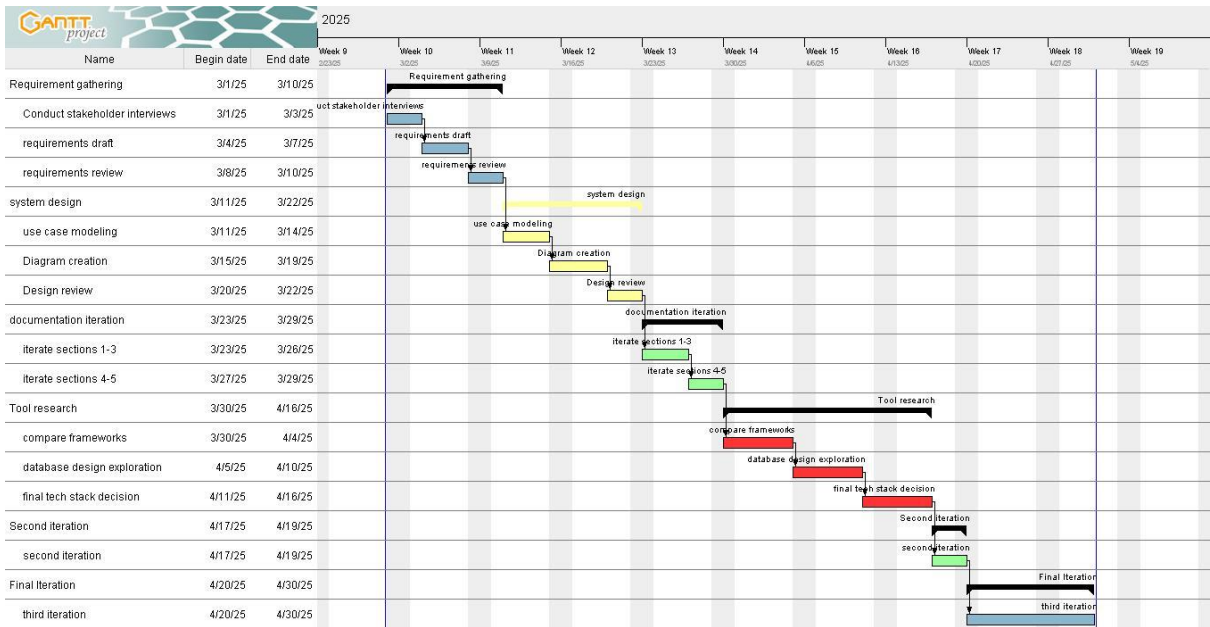
Introduction written by Eireneos. 

Table 1: Tasks schedule breakdown

Task Category	Subtask	Start Date	End Date	Dependencies
Requirement Gathering	Conduct stakeholder interviews	01/03/2025	03/03/2025	—
	Write requirements draft	04/03/2025	07/03/2025	Interviews
	Requirements review & finalization	08/03/2025	10/03/2025	Draft
System Design	Use case modeling	11/03/2025	14/03/2025	Requirements

	Create diagrams (DFD, UML, Sequence)	15/03/2025	19/03/2025	Use Cases
	Final design review	20/03/2025	22/03/2025	Diagrams
Documentation Iteration 1	Iteratively over sections 1–3	23/03/2025	26/03/2025	Requirements
	Iterate over sections 4–5	27/03/2025	29/03/2025	Design
Tool Research	Compare frameworks (React, Django, etc.)	30/03/2025	04/04/2025	—
	Database design exploration	05/04/2025	10/04/2025	Requirements
	Final tech stack decision	11/04/2025	16/04/2025	Comparisons
Documentation Iteration 2	Second iteration(Before pre-Easter review)	17/04/2025	19/04/2025	Feedback from mentors
Documentation Iteration 3	Third iteration based on feedback over 1-5.	20/04/2025	30/04/2025	Tech stack & testing info
	Adv1: Review requirements			
	Adv2: Review Gantt Chart			
	Adv3: Review Use Cases			
	Adv4: Review Diagrams			
Implementation & Testing	This will happen next semester.	-	-	-

Gantt Chart by Andreas Karpasitis



The new gantt chart is a continuation of the old one, the new one is for the creation of the website.

Figure 2: Gantt Chart

The gantt chart has been made with a free application called GanttProject (which is fairly easy to use and also free to export the gantt charts you make into pdf or jpeg and so on).

MONITORING AND CONTROLLING MECHANISMS

Section 2.3 Monitoring & Controlling Mechanisms, by Peter M

When it comes to managing the project progress, we relied on an independent and deadline-based model for completing various milestones that we set since the start of the semester.

Notably, our group wanted to avoid micromanaging one another or holding regular meetings. We decided to implicitly trust one another to be responsible for their tasks and sections without micromanagement, but we communicated clearly that if anybody is stuck and they would like to ask for help, they most definitely should.

We had minimal meetings, but we communicated almost entirely through text on **Discord**. In general, Discord was used extensively to manage the project progress for the document (which was imported and backed up on **Google Docs**), because we agreed that having a **chat history** that we can go back to and check would be more useful and less headache inducing than a meeting. **“If something can be done over text, do that instead”**.

Because of this, we had an **asynchronous** process where any teammate could talk to any other without a need for preamble; we can simply text each other at any time when we need something without requiring scheduled meetings - despite that, we still had two of them to synchronize our expectations and vision.

To summarize:

PROJECT DOCUMENTATION & COLLABORATION

- **Google Docs:** Used as the primary tool for collaborative writing, documentation, and tracking project progress. A side copy is maintained as a backup to prevent data loss.
- **Version History in Google Docs:** Enables tracking of changes and allows rollback to previous versions if needed.

COMMUNICATION & REPORTING

- **Discord:** The main communication platform for discussions, updates, and coordination.
- **Text-Based Problem Solving:** Issues are primarily resolved through Discord messages to maximize efficiency.
- **Asynchronous Updates:** Team members provide updates and share progress in chat channels to keep everyone informed without requiring scheduled meetings.

RISK MANAGEMENT & ISSUE TRACKING

- **Issue Logging in Google Docs:** We used comments to discuss any issues, challenges, or roadblocks on the google document itself, and any further details about it were discussed on discord.

- **Backup Copy:** A duplicate of the project documentation is maintained to ensure information is not lost due to accidental changes.

Google Doc Comments!

restaurant details in morning.

Section 2.3 End

The Payment Enrollment System must follow a **three-tier layered architecture**, ensuring modularity, scalability, and maintainability. The system should logically consist of the following layers:

1. Presentation Layer (Frontend):


Peter Maravgakis
 7:39 PM Mar 30

Rewrote these today.

And Discord communication!



March 28, 2025


 3/28/2025 9:22 PM
 Hey guys @everyone if you guys want and can we are at coffee berry near the university if you wanna join us


 3/28/2025 10:40 PM
 Had meeting 1 to get a general review of our work, here's the notes:

- We need to show the document next week on the 3rd of the month for feedback from the Professor and other groups
- On the document we've filled out sections 1, 3, & 4 (Go to the doc and ask

finance department to prepare some audits so that the payments are all correct and legal.

Address

3.2 Functional and Non-Functional Requirements

Define the system's capabilities and constraints |

3.2 This is going to be an early draft. Comments I whenever you feel comfortable with updating 3.1, please!

Functional Requirements

- HR Personnel and employees must login into the system to access any of its functionalities
- HR Personnel can register new employees onto the database of employees
- HR Personnel can edit the employees' existing information
- HR Personnel can use the website to generate financial records (payslips, deductions, leaves, salary reports)
- Financial Personnel will be able to check with the system the overall payments done by the system to the employees over the months for auditing purposes
- Employees (and basically everyone registered onto the system and thus authorized) can view the financial records
- Management should be able to access the database and update and or backup manually if anything would seem to fail like the electricity going out
- The system should be able to flag conflicts occurring in the system to the management
- It should also be able to send out emails to employees regarding payment, leaves and for management for approval of leaves

Req ID	Description	Use Case (Section 5.1)
FR-01	Authenticate users via credentials + 2FA.	UC1 (Employee Onboarding)


 2:47 AM Today

Add caption to your figure and use it to reference to it in your report. Where is the Level 1 DFD diagram?

Tab 1 - The Finance Team uses the s...


Constantinos Stephanides
 2:43 AM Today

Add a small discussion paragraph for each diagram, AC1 to AC4.

Tab 1 - This will be integrated in the ...


Andrew Karpas
 3:40 AM Yesterday

Please provide a mapping between the mentioned standards and the different components of your system that they apply - comment from the professor

Tab 1 - FR-02


Peter Maravgakis
 8:10 PM May 22

Draft 3. I think structuring it like this removes some redundancies

Section 2.3 End

Final Iteration by Andreas

REQUIREMENTS AND SPECIFICATIONS

The requirements have been discussed with the group during meetings to try to find the most important requirements for our system. The requirements are the things we need to apply to our

system so that we can make it work as efficiently and as best as possible. In doing this we ensure as close as possible to a working system that has the fundamental processes that will help the user and the engineer to make the web app as functional and easy to comprehend as possible.

As a general overview of the rest of these sections, here are the general requirements of the system:

- Hold the information of all the employees
 - it needs to be able to hold and edit personal information
 - being able to generate employee ID for each employee
 - needs to have a record of the leaves taken by each employee
 - the working hours
- keep records in a secure database
 - will need to keep these records in a database that is secure enough not to be accessed by outsiders
 - database should be big enough to hold enough information about the employees and their salaries
 - should also be able to calculate the salaries according to leaves, working hours etc.
- the system should be able to output a payslip for each employee
 - payslip should be produced for each month according to the salaries
 - should be able to clearly output leaves and or other forms of payment deductions (eg tax, sick leaves etc.)
- secure login
 - during the login step to be able to validate the login credentials (password and ID with the database versions to cross check)
- website interface for employees and different for management
 - the website should be able to have different interfaces for different categories of employees
 - the basic employees will have a basic interface
 - the management will be able to have a more in depth interface which has access to more detailed aspects of the system
- conflict checking for inputting different leaves (extending past the allowed amount) (check for leaves that maybe overlap between employees)
 - the system should be able to flag the management side of the system that a conflict in the leaves section is taking place
 - if the leaves requested by an employee are more than the amount that is allowed then the system should be able to send a notification or flag the person to ensure that the right amount of leaves are requested for each employee
- send email reminders
 - the system should be able to send emails to management about payments and the payment slips
 - also should be able to send email reminders about leaves that have already been approved
 - should be able to send emails about pending leaves to the management to be approved
 - should also send email confirmations of received payment
- daily backups

- system should be able at the end of the day to backup the files to the database with the new information it has received
- detailed salary report
 - apart from the payslip the system should also be able to produce an in depth salary report which will indicate how the hours have been calculated and how the tax deductions and the GESY have been calculated also so that the employee clearly knows the overall deductions and the work hours he has worked and on which days without needing for the employee to keep track of everything himself
- working website on all systems
 - the website should be able to work on different browsers as well as on mobile phones

STAKEHOLDERS OF THE SYSTEM

The payroll system will have a few number of stakeholders which will be:

- End-users (employees): The employees will be able to see their salary from a pay slip which will be generated from the system which will be calculated from the leaves, tax , social insurance and days worked. Also the employee will be able to see how many paid leaves he has left to take.
- System Administration: the system admin will be able to oversee the overall system and will be able to maintain it as well as keep the system as secure as possible. The SA should also allow for the application to be as smooth as possible with third party cooperation such as banks and tax authorities so that the system works without any issues.
- Human resources: The HR department will be able to oversee the payments made by the system to include maybe some added bonuses and also keep a record of the salary records of the employees. Also if an employee gets a raise the HR department should be able to update the records of the siad employee to receive from the system the correct amount in the monthly payment from then on.
- Financial personnel: The financial department will be responsible to audit the payments of the system. This means for the 3 month mark to be able to check the expenses of the system (employee payments and other expenses) so that the accounting part of the system is carefully monitored to abide by national law. This is done by checking the monthly payments of the employees and doing calculations to figure out the overall cost each 3 month period.

Andreas

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Define the system's capabilities and constraints.

3.2: This is going to be an early draft. Overwrite it whenever you feel comfortable with updating it. -Peter M

Functional Requirements

- HR Personnel and employees must login into the system to access any of its functionalities
- HR Personnel can register new employees onto the database of employees
- HR Personnel can offload employees

- HR Personnel can edit the employees' existing information
- HR Personnel can use the website to generate financial records(payslips, deductions, leaves, salary reports)
- Financial Personnel will be able to check with the system the overall payments done by the system to the employees over the months for auditing purposes
- Employees (and basically everyone registered onto the system and thus authorized) can view the financial records
- Management should be able to access the database and update and or backup manually if anything would seem to fail like the electricity going out
- The system should be able to flag conflicts occurring in the system to the management
- It should also be able to send out emails to employees regarding payment, leaves and for management for approval of leaves

Req ID	Description	Use Case (Section 5.1)
FR-01	Authenticate users via credentials + 2FA.	UC1 (Employee Onboarding)
FR-02	Assign unique employee IDs during onboarding.	UC1 (Employee Onboarding)
FR-03	Calculate net salary (gross pay - deductions).	UC3 (Payroll Processing)
FR-04	Generate PDF payslips and detailed salary reports.	UC5 (View/Download Payslips)
FR-05	Submit/approve leave requests; flag conflicts.	UC2 (Leave Management)
FR-06	Interface with banking APIs for salary transfers.	UC4 (Payment Disbursement)
FR-07	Provide role-specific UIs (employee vs. HR).	UC1, UC2, UC5
FR-08	Send email notifications for approvals/reminders.	UC2 (Leave Management)

Table 2: Functional Requirements & Corresponding Use Cases

The nonfunctional requirements are requirements in which the system as a whole will need to act upon without having the user seeing what the functionalities are, for example the user does not need to know how many users the system can hold without having any issues. The process in which this happens is through the functional requirements some issues may arise and cause problems so in this way we will need to work on the things that will cause issues that do not affect the visual aspect of the system and these are:

Nonfunctional Requirements

- All entry fields must have validation so that the correct data types are entered(names, dates, money etc)
- The system should be able to send out notifications when updates to financial records are made
- The system should be able to handle a concurrent number of users(**Ballpark figure**, proportional to how big the company using it is, say, a 100-person company can expect 20 users to be concurrently using the website)
- The system should be able to process multiple function calls to generate financial records at once without crashing(**Ballpark figure**: 100-person company, 20 concurrent users, **5** CRUD calls at any given time?)

SYSTEM ARCHITECTURE

Section 4: System Architecture, by Peter M. 

The Payment Management System must follow a **three-tier layered architecture**, ensuring modularity, scalability, and maintainability. The system should logically consist of the following layers:

1. Presentation Layer (Frontend):

- a. Provides an intuitive user interface for HR personnel, employees, financial personnel and system admins to interact with the system.

- b. Developed using modern web technologies such as React.js, HTML5, Angular or similar.

2. Logic Layer (Backend):

- a. Handles core functionalities such as salary computation, tax deductions, and payment processing. The entirety of the business logic is here, and anyone using the frontend needs to pass from this layer to read/write data to the data layer.
- b. On top of read/write queries, it can also handle complex functions such as sorting
- c. Implemented using the Node.js framework

3.

Data Layer (Database):

- a. Stores employee records, payment history and financial reports.
- b. Utilizes the relational database MySQL for secure and efficient data management.

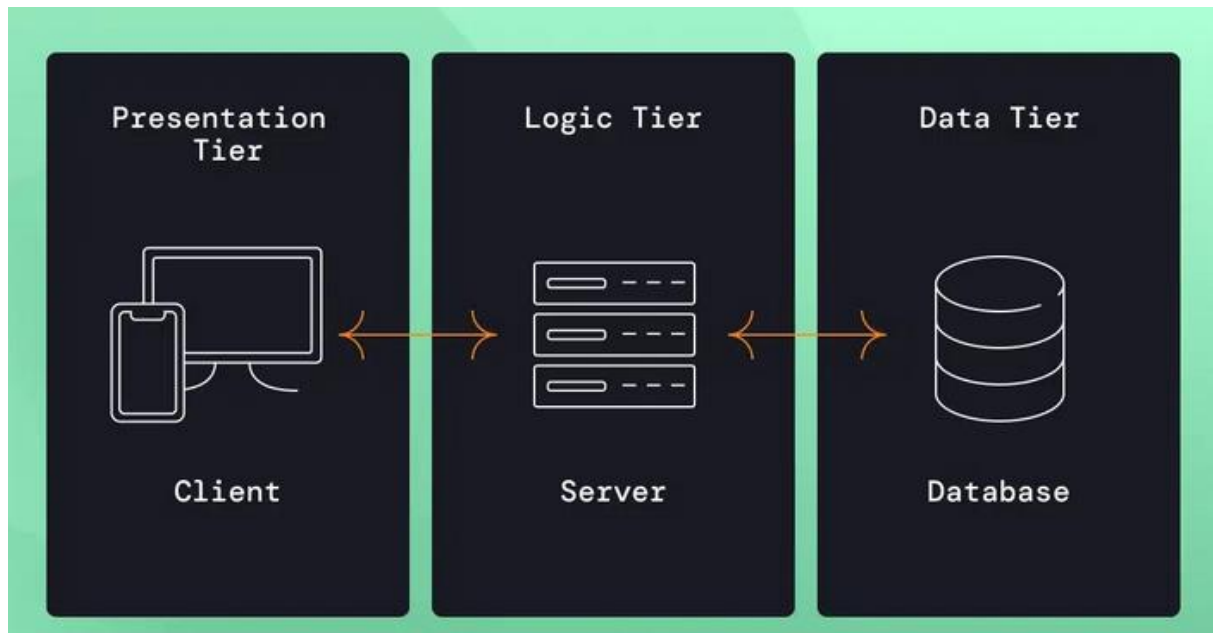
System Workflow:

1. HR inputs employee details, including salary structure and tax information.
2. The backend calculates net salaries, deducts taxes, and updates financial records.
3. Payments are processed via integrations with banking systems.
4. Employees can access their salary details and payslips through the frontend interface.

This architecture ensures secure data handling, seamless integration with third-party payment providers, and high availability for payroll management.

It essentially covers all the use cases of a CRUD-based application while also being highly modular to allow its enrollment across all kinds of devices.

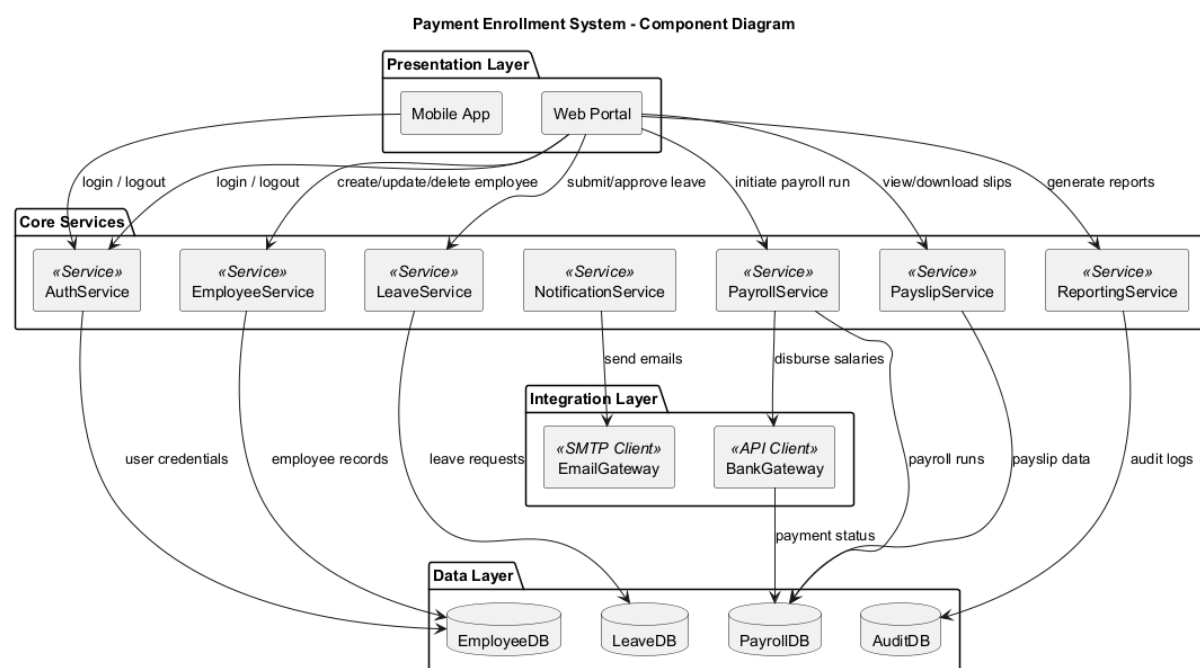
HIGH-LEVEL DEPICTION OF PES



Section 4 Fig 1:

A high-level representation of the system.

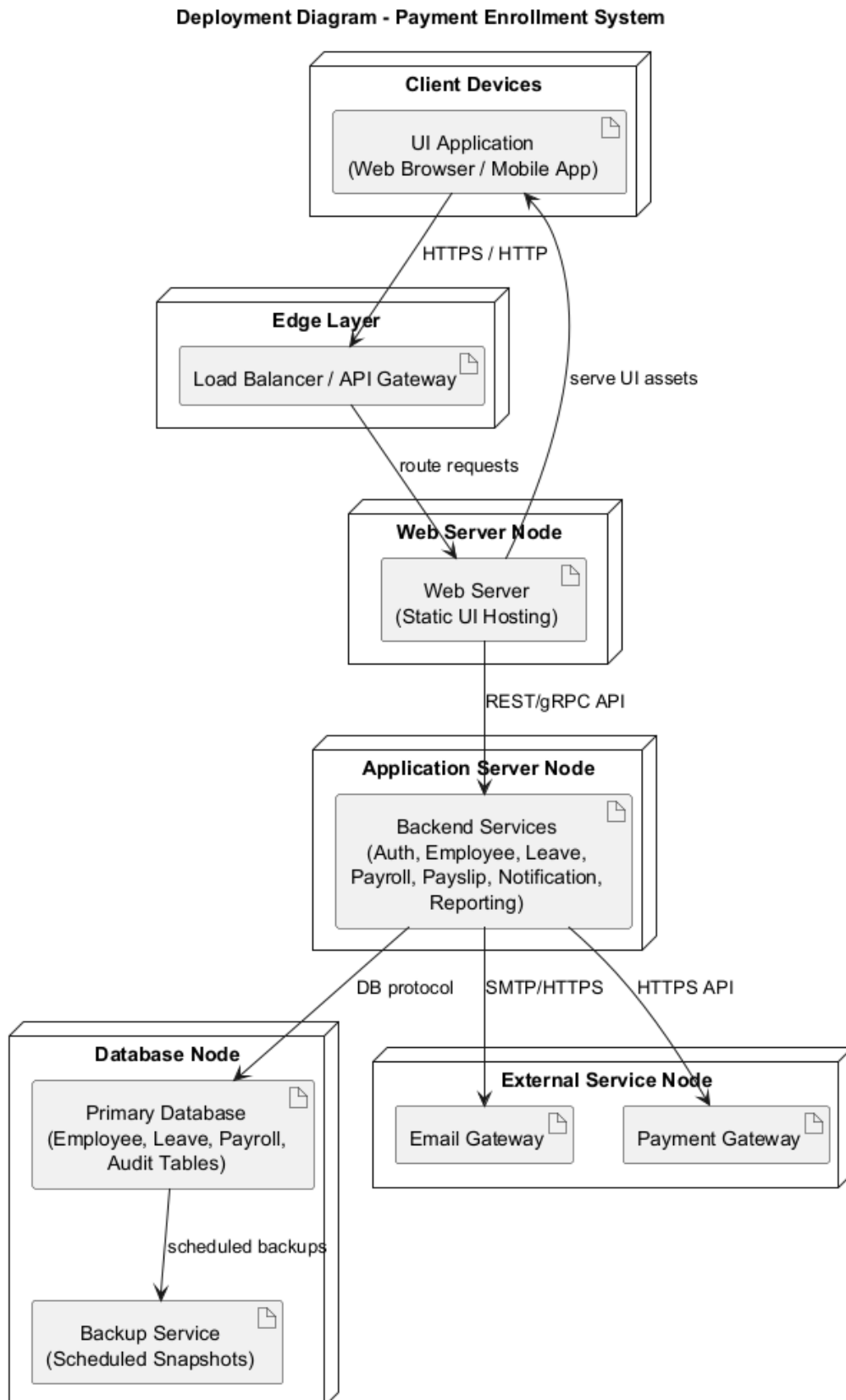
UML COMPONENT DIAGRAM



Section 4 Fig 2:

The Component Diagram for PES, by Peter M.

The UML Component Diagram demonstrates the interfaces that each component of the PES provides and requires. It is important that each individual module is implemented separately to make it easier to scale or switch out modules as needed. In this stage of development(May 2025), the application stack has not yet been decided on which makes this diagram doubly important.



Section 4 Fig 3: The Deployment Diagram for PES, by Peter M.

A UML Deployment Diagram shows the nodes of the PES, whether they're virtual or physical, as well as the executable artifacts deployed to them, and the communication paths between nodes. Given that the PES is a write/read system with a logic and database layer that deals in very little other than

text saving and text manipulation, it stands to reason that it can be easily deployed on a single computer.

Keeping it on a single computer actually carries many positives when it comes to data consistency, as we do not have to worry about the typical things a distributed system needs to worry about, like database replication and write/read consistency issues.

Section 4 End

SYSTEM DESIGN

This section includes detailed design elements such as use cases, diagrams, and traceability matrices.

USE CASES

Use case 1: Login page

Actors: HR Personnel, Employees, Management, Financial team

Preconditions: Personnel should be in the database to be able to login successfully

Basic Flow:

1. The employee of the company, either HR or manager or at any position is in the login page of the web page
2. Enters their credentials provided to them by their manager or HR
3. Database will authenticate and check for the correct information of the person
4. If the credentials are not correct then the person is not granted access
5. If the credentials are correct then the employee can move into the site

Use Case 2: Employee Onboarding

Actors: HR Personnel, System Administrator

Preconditions: The HR personnel must have the necessary credentials to access the system.

Basic Flow:

1. The HR personnel logs into the system.
2. The system verifies the credentials from the database and grants access.
3. The HR personnel selects the "Add New Employee" option.
4. The system prompts for employee details (Name, Contact Info, Designation, Department, Salary, Tax Brackets, Benefits, Deductions, etc.).
5. The HR personnel enters the required information.
6. The system validates the data and stores the employee record in the database
7. A confirmation message is displayed.

Alternative Flows:

- If required data is missing, the system prompts the user to complete all fields.
- If the HR personnel lacks permissions, access is denied.

```
Function OnboardEmployee(hrCredentials, employeeData):  
    If Authenticate(hrCredentials) is False:  
        Return "Authentication failed"  
        Break  
    If HasPermission(hrCredentials, "add_employee") is False:  
        Return "Access denied"  
        Break  
    If IsComplete(employeeData) is False:  
        Return "Missing required fields"  
        Break  
    If AddEmployeeToDatabase(employeeData) is False:  
        Return "Failed to save employee record"  
        Break  
    Return "Employee successfully onboarded"
```

PSC1: Onboard Employee Function by Constantinos

For UC 1 and 2

Use Case 3: Attendance recording

Actors: Employees, HR Personnel, System, Database

Preconditions: Management should have access to the attendance records of their employees

Basic Flow:

1. Attendance is recorded by the Manager and updated every week
2. Manager logs into the system and goes to attendance

3. If an employee has missed a day without notice it is noted down as an absence
4. With the absence recorded the payroll will be calculated accordingly from the system
5. Leaves will also be tracked by the manager and not be written as absent but on leave to have the correct payment at the end of the month

Alternative Flows:

- If an employee forgets to log attendance, HR can manually update records.
- If leave exceeds available balance, the system notifies the employee.

Use case 4: Leave request

Actors: HR Personnel, Employees, Management, Financial team

Preconditions: Limited number of leaves can be taken by each employee

Basic Flow:

1. Employee logs into the system
2. Goes to request for leave
3. The employee should request the number of days he is going to be on leave
4. If the number of days is more than the accepted value then an error message should let them know
5. If the leave number is accepted then the leave request should be sent to the manager (Use case 4) to be accepted

6. Leave balance should be updated after the request is accepted

Use case 5: Leave management

Actors: Management

Preconditions: Management needs to have access to the attendance to be able to accept or deny leaves

Basic Flow:

1. Management logs into the system
2. Goes into the leave requests and sees pending requests of the employees
3. Manager can check from the attendance or other requested leaves and decide if he can accept or reject
4. If accepted the attendance calendar will be updated with the accepted leaves on it

Alternative Flows:

- If the leave is a sick leave then the employee will be requested to provide the relevant documentation

```
Function ManageAttendanceAndLeave(employeeCredentials, leaveRequest):  
    If RecordAttendance(employeeCredentials.id, TodayDate) is Null:  
        ForwardToHR(employeeCredentials.id, TodayDate)  
  
    If SubmitLeaveRequest(employeeCredentials.id, leaveRequest):  
        RequestToHR(employeeCredentials.id, leaveRequest)  
  
    If leaveRequest.days > GetLeaveBalance(employeeCredentials.id):  
        ForwardToEmployee(employeeCredentials.id, availableBalance)  
    Else  
        UpdateLeaveBalance(employeeCredentials.id, daysOff)  
        UpdatePayroll(employeeCredentials.id, daysOff)
```

PSC: Attendance & Leave Management by Constantinos.

For UC 3,4,5

Use Case 5: Payroll Processing & Salary Calculation

Actors: Finance Team, System

Preconditions: Employee records, attendance, and leave data must be up to date.

Basic Flow:

1. The finance personnel logs in and selects "Process Payroll."
2. The system retrieves employee records, attendance, and leave data.
3. The system calculates gross salary, deductions (taxes, social insurance, etc.), and net pay.
4. The system generates a payroll summary report.
5. The finance personnel reviews and confirms payroll processing.
6. Payroll data is stored for compliance and auditing.

Alternative Flows:

- If an employee record is incomplete, an error message is displayed.
- If payroll approval is needed, it is sent for managerial authorization.

```
Function ProcessPayroll(financeCredentials):  
  
If IsPayrollRecordComplete(employee.id) is NULL:  
    LogError(employee.id, "Incomplete record")  
else  
    gross = CalculateGrossSalary(employee.id)  
    deductions = CalculateDeductions(employee.id)  
    net = gross - deductions  
  
    If Payroll.approval == True:  
        ApprovePayroll(employee.id, net)  
        Display Message "Wait for approval"  
    Else  
        payroll = GeneratePayroll(employee.id)  
        StorePayrollData(employee.id, payroll)
```

PSC3: Payroll Process by Constantinos.

Use Case 6: Payment Disbursement

Actors: Finance Team

Preconditions: Payroll must be processed and approved.

Basic Flow:

1. The finance personnel initiates salary disbursement.
2. The system compiles payment details and interfaces with the database.
3. The system updates payment status and generates a transaction log.
4. The user receives confirmation of successful payment processing.

Alternative Flows:

- If payment fails, the system logs the issue and alerts the finance team.

```
Function DisbursePayments(financeCredentials):  
  
    if Authenticate(financeCredentials) is False:  
        Return "Authentication failed"  
        Break  
  
    if IsPayrollApproved() is False:  
        Return "Payroll not approved"  
        Break  
  
    payment = CompilePaymentDetails()  
    response = BankAPI.Send(payment)  
  
    if response == "Failure":  
        LogIssue(payment.id, response.error)  
        NotifyFinanceTeam(payment.id)  
        Continue  
    else if response == "NoResponse":  
        ScheduleManualIntervention(payment.id)  
        Continue  
    else  
        UpdatePaymentStatus(payment.id, "Success")  
        LogTransaction(payment)  
  
    Return "Payment disbursement completed"
```

PSC4: Payment Disbursement by Constantinos.

Use Case 7: Viewing and Downloading Pay Slips

Actors: Employees, HR Personnel, Manager, Finance Team

Preconditions: Payroll must be processed.

Basic Flow:

1. The employee logs into the system.

2. The system verifies credentials and grants access.
3. The employee navigates to "Pay Slips."
4. The system displays available pay slips.
5. The employee selects a pay slip to view or download.
6. The system generates a PDF version for download.

Functionality:

User Authentication module: Verifies employee, HR or manager credentials before granting access. Ensure secure password handling and session management.

Role-Based Access Control: Restricts access HR can view all employees slips, Managers can view only their team employees. Add explicit permission checks for HR vs Manager vs Employee.

Fetch Pay Slips Function: Retrieves pay slip record for the authenticated user from the database. Optimize database queries, handle empty result.

PDF Generation Module: Converts selected pay slip to a PDF for download.

Download Handler: Triggers and manages the file download process. Improve error handling to ensure correct MIME type for PDFs.

User interface: Allows user to navigate, view available slips and select for download, include role-based filtering and modern UI styling.

Error & Notifications System: Display messages such as "No pay slips available" or "Download Failed" and include retry options.

```

Function ViewAndDownloadPaySlip(employeeCredentials, slipID):

    If Authenticate(employeeCredentials) is False:
        Return "Authentication failed"
        Break

    slips = FetchPaySlips(employeeCredentials.id)

    If slips == NULL:
        Return "No pay slips available"
        Break

    If slips.ID == NULL:
        Return "Requested slip not found"

    If GeneratePaySlipPDF(slips.ID) is False:
        Return "Failed to generate PDF"
        Break

    If TriggerDownload(slips.ID) is False:
        Return "Download failed"
        Break
    else
        slipFile = GenerateSlipPDF(slips.ID)
        Return "Pay slip downloaded successfully"

```

PSC5: Viewing & Downloading Pay Slips, by Constantinos.

Use Case 8: Compliance and Financial Reporting

Actors: Finance Team

Preconditions: Payroll and financial transactions must be recorded.

Basic Flow:

1. The finance team logs in.
2. The system verifies credentials and permissions.
3. The user navigates to "Reports" and selects a reporting category.
4. The system retrieves and compiles the relevant data.
5. The user reviews and exports the report.
6. The system logs the report generation activity for security tracking.

Alternative Flows:

- If no data is available, the system notifies the user.
- If unauthorized access is attempted, an alert is triggered.

```

Function GenerateComplianceReport(userCredentials, reportType):

    if Authenticate(userCredentials) is False:
        Return "Authentication failed"
        Break

    if HasPermission(userCredentials, "view_reports") is False:
        TriggerAlert("Unauthorized access")
        Return "Access denied"
        Break

    data = FetchReportData(reportType)

    if data == NULL:
        Return "No data available"
    else
        dataExport = GenerateExport(data)
        UpdateLogReport(user.id)
        Return "Report generated successfully"

```

PSC6: Compliance & Financial Reporting by Constantinos.

Use Case 9: Update salary

Actors: Managers, HR

Preconditions: Payroll and financial transactions must be recorded.

Basic Flow:

1. Managers or HR login to page
2. Go to update salary
3. Enter the details of the employee that requested a raise
4. Page indicates the latest salary of employee
5. Manager or HR enters the new amount
6. System updates the new salary

Alternative Flows:

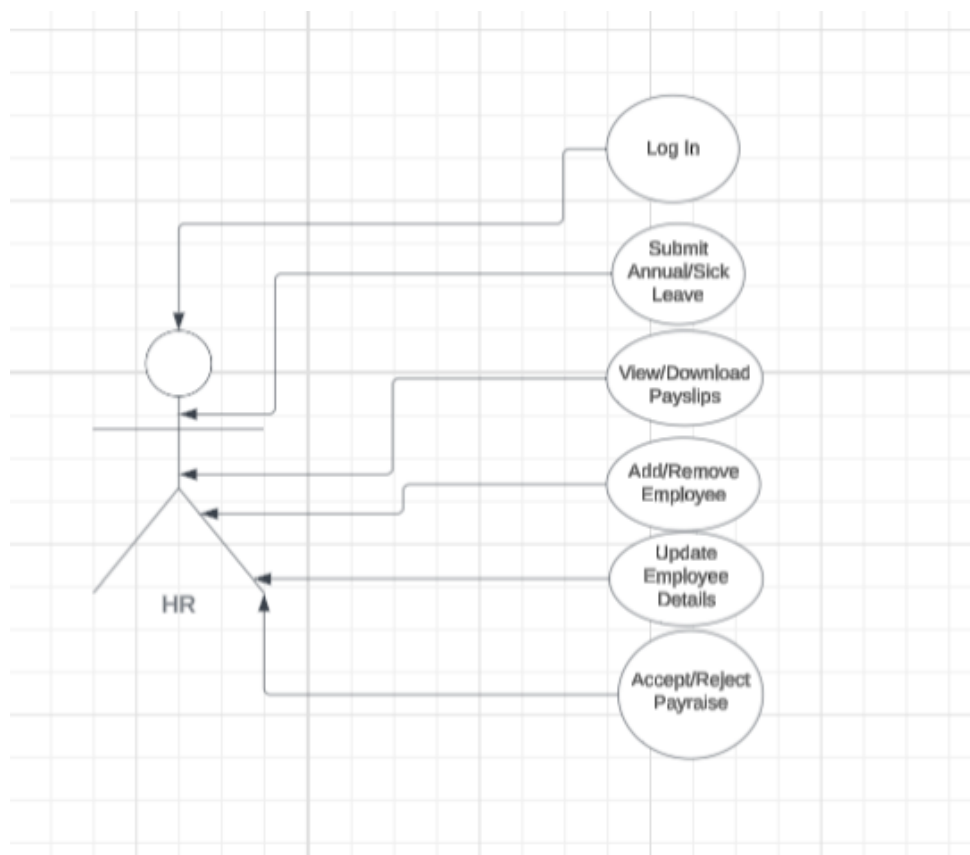
- If no data is available, the system notifies the user.

5.1: This is going to be an early draft. Overwrite it whenever you feel comfortable with updating it. -Peter M

The diagrams illustrate the key actors and their respective interactions with the Payroll and HR Management System. Each actor **HR Personnel**, **Employee**, **Finance Team** is linked to a specific set of actions that reflect their role and responsibilities within the system. These include essential operations such as logging into the platform, managing attendance, onboarding employees, processing payroll, disbursing salaries, reviewing financial transactions, and exporting reports. By visually segmenting each actor's allowed operations, the diagrams clarify role-based access and ensure that system functionality is both organized and secure. This structured representation helps support accurate requirements definition, access control design, and effective communication across development, administrative, and audit teams.

Actor Centric Use Case 1: HR Personnel Actions

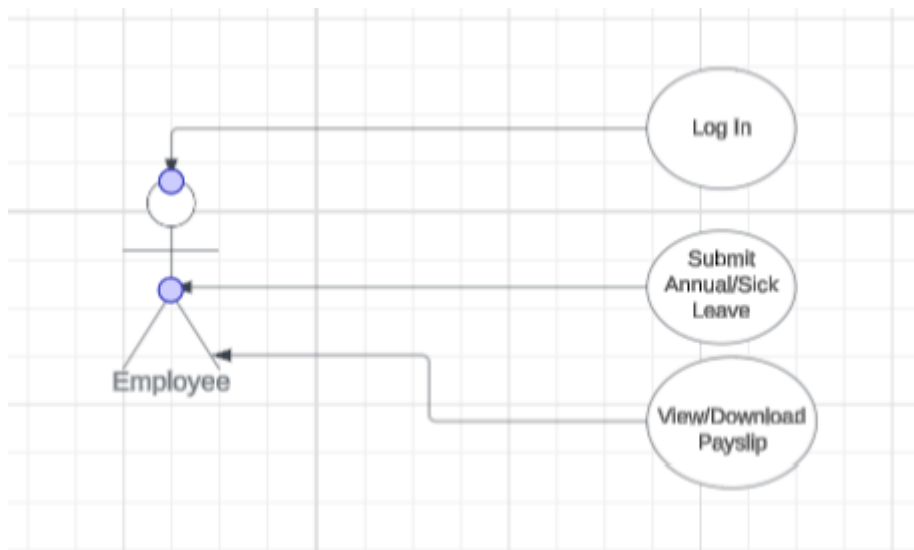
HR Personnel are responsible for managing employee records and overseeing daily HR operations. They log in to onboard new staff, review leave requests, update attendance records, and access or generate payslips. These actions support workforce tracking and ensure that employee data remains current and accurate. Their involvement is essential for maintaining efficient administrative workflows.



AC1: HR Personnel System Actions(Constantinos)

Actor Centric Use Case 2: Employee Actions

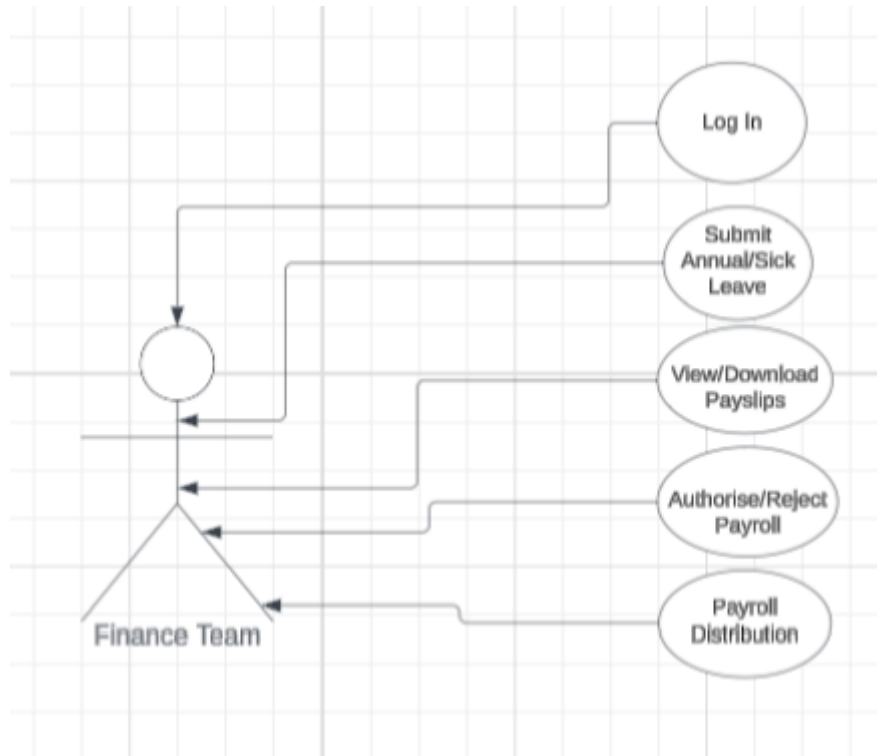
Employees interact with the system to manage their attendance and HR-related requests. They can log in, mark their attendance, submit leave applications, and view or download their payslips. These functions empower employees to take responsibility for their records while reducing the need for direct HR intervention. It simplifies communication and increases transparency in personal data access.



AC2: Employee System Actions(Constantinos)

Actor Centric Use Case 3: Finance Team Actions

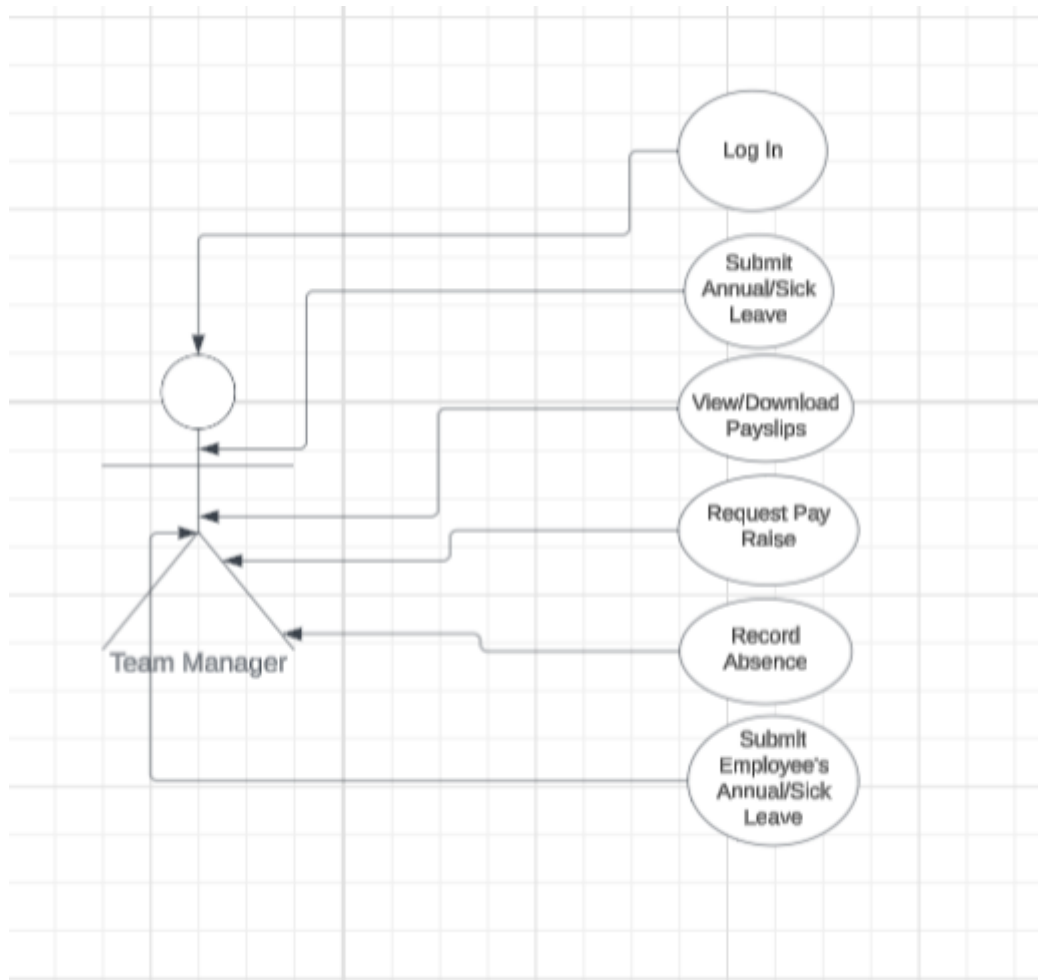
The Finance Team uses the system to handle payroll and financial reporting. After logging in, they review payroll reports, process salary disbursements, and check transaction records. They also export financial data as needed for audits or reporting. Their tasks ensure financial accuracy and timely salary processing across the organization.



AC3: Finance Team System Actions(Constantinos)

Actor Centric Use Case 4: Team Manager Actions

The Team Manager uses the system to oversee employee attendance, leave, and payroll-related activities. After logging in, they can submit their own annual or sick leave, view and download payslips, and request pay raises when applicable. Additionally, they are responsible for recording employee absences and submitting annual or sick leave on behalf of their team members. These actions help maintain accurate attendance records and ensure fair management of staff leave and payroll processes.



AC4: Team Manager's System Actions(Constantinos)

DATA FLOW DIAGRAM (DFD)

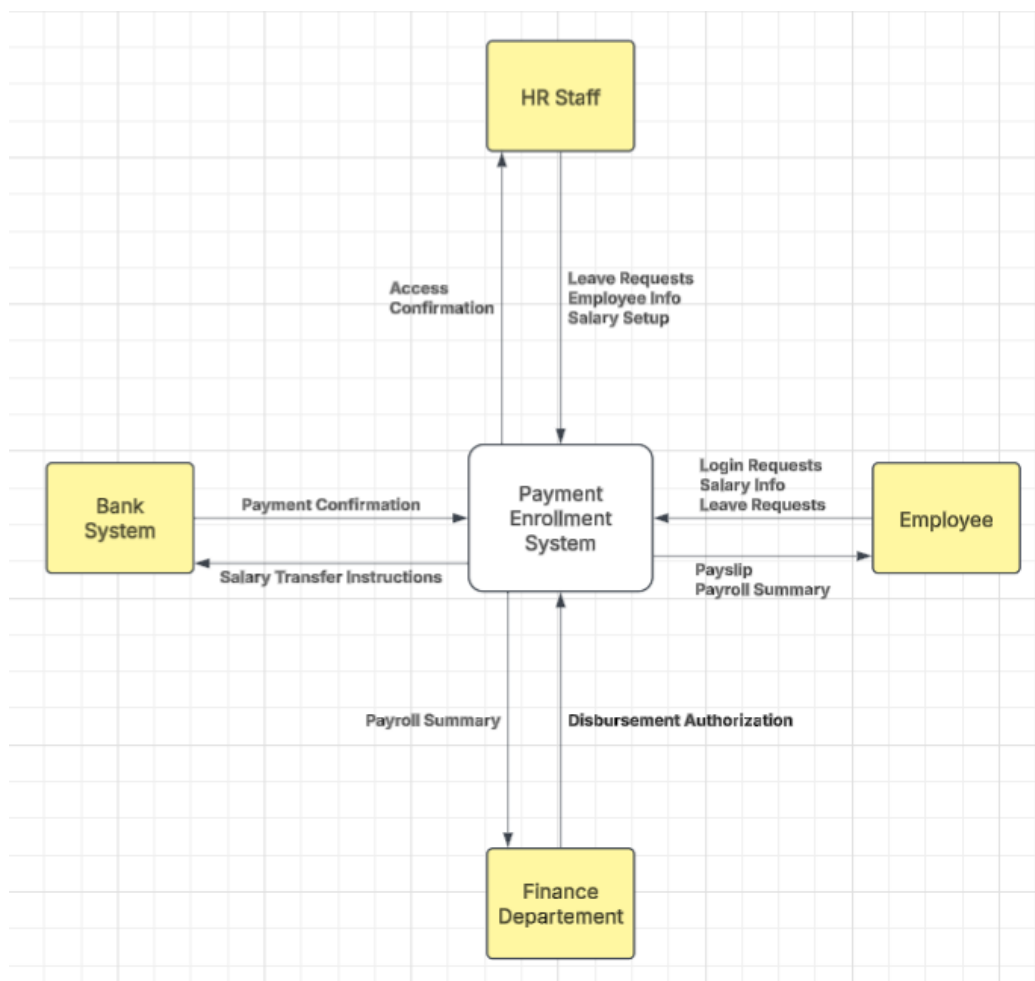
Constantinos

The **Level 0 Data Flow Diagram** represents the overall interaction between the **Payment Enrollment System** and its external entities: **HR Staff, Employees, Team Managers** and the **Finance Department**.

- **HR Staff** interact with the system to input employee information and configure salary details. They can also submit or manage leave requests. The system confirms access rights and updates employee records accordingly.
- **Employees** use the system to log in, submit leave requests, and view their payslips. The system also processes their salary data and makes it accessible in a clear, downloadable format.
- The **Finance Department** receives summarized payroll information from the system to review and authorize disbursements. Once approved, they send a **disbursement authorization** back to the system.

- Team managers

The diagram is essential during the early stages of development, as it helps define system boundaries, clarify data responsibilities, and guide the decomposition into more detailed processes. It will be used throughout the project to align implementation with functional expectations and ensure complete coverage of all stakeholder interactions.

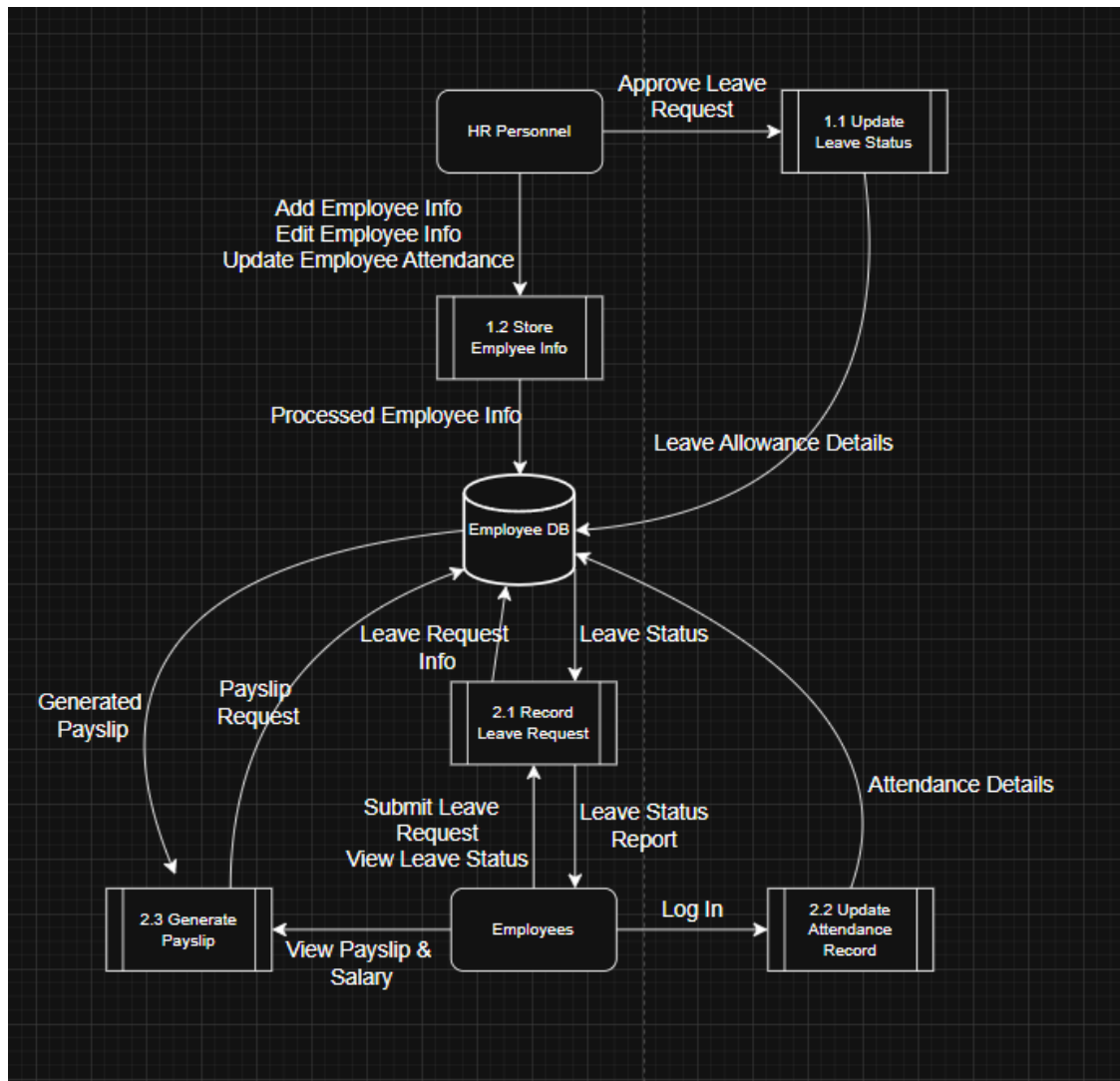


Level 0 DFD

Level 1 Data Flow Diagram: HR Personnel & Employees Processes

This Level 1 Data Flow Diagram serves as a detailed decomposition of the Employee and HR-related interactions outlined in the Level 0 DFD for the Payment Enrollment System. It breaks down the single high-level process into focused subprocesses that handle core HR functions such as storing employee records, updating attendance, and approving leave requests, as well as employee-driven actions like submitting leave, viewing payslips, and checking leave status. All these processes interact with a central Employee DB, which serves as the primary data store for personnel, attendance, and payroll-related information.

To ensure clarity and maintain a clean visual structure, this diagram focuses exclusively on the interactions between Employees and HR Personnel. A second Level 1 DFD will separately present Finance and Bank related processes.

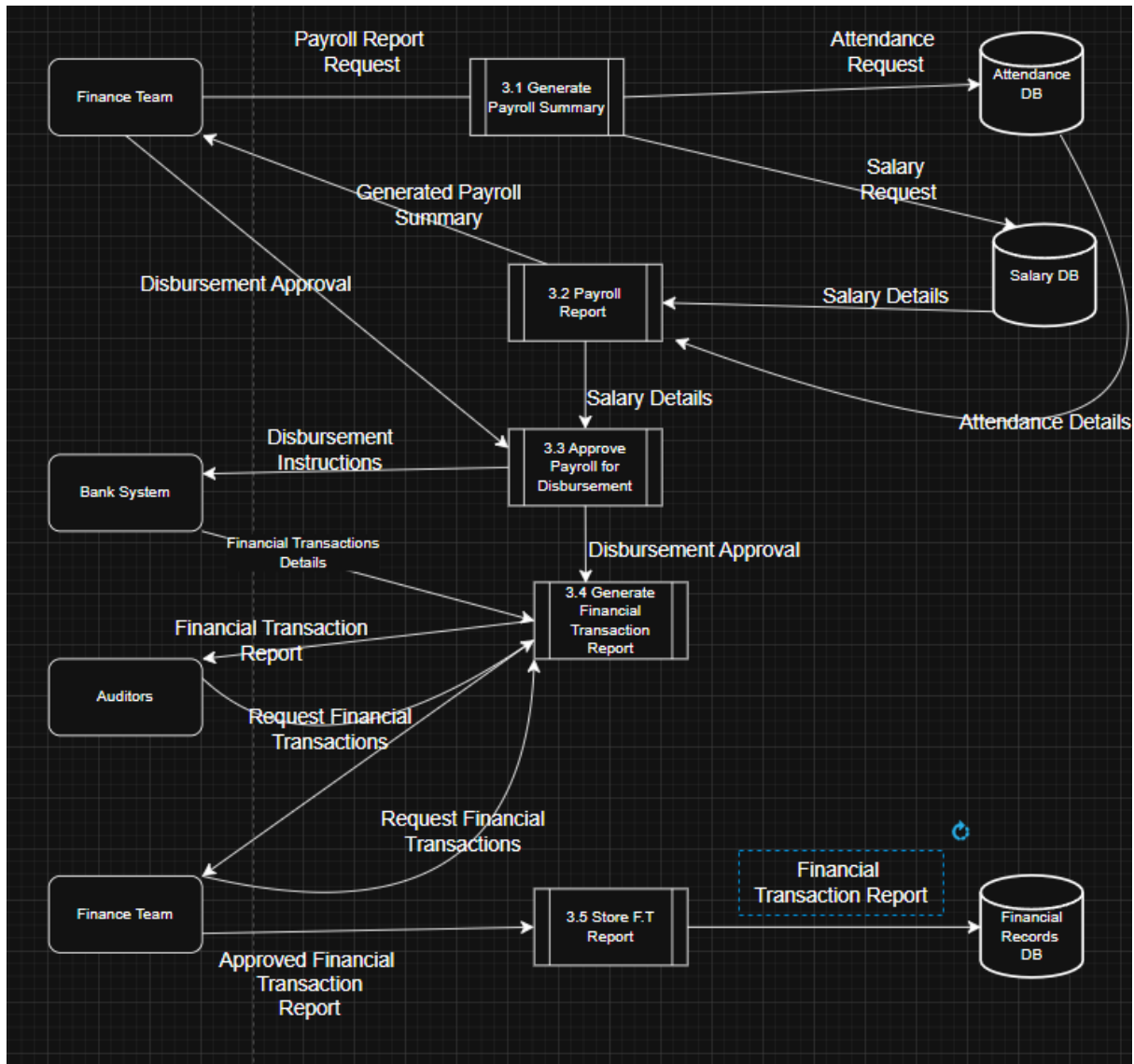


Level 1 DFD : HR Personnel & Employees Processes

Level 1 Data Flow Diagram: Finance Team Processes

This Level 1 Data Flow Diagram expands on the financial and audit-related operations from the Level 0 DFD of the Payment Enrollment System. It details how the Finance Team generates payroll summaries, approves disbursements, and initiates salary transfers through the bank. The Bank System returns payment confirmations, which are processed into financial transaction reports.

Auditors and finance personnel can request these reports, which are generated and stored in the Financial Records DB. All processes are integrated with relevant data stores to ensure consistency and traceability. This diagram is presented separately from the HR and Employee view to maintain clarity and focus across system domains.

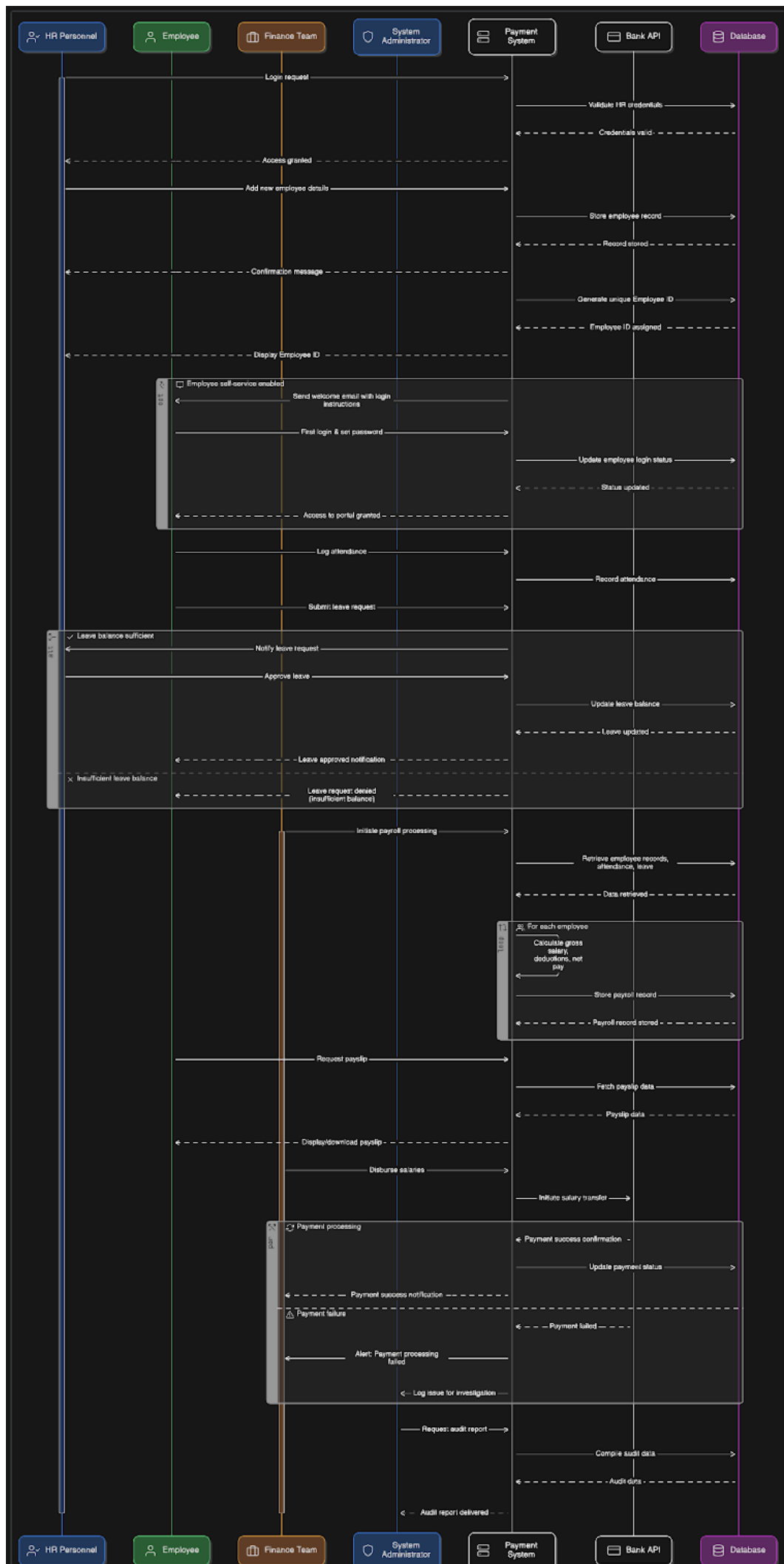


Level 1 DFD: Finance Team Processes

Level 3 Data Flow Diagram: PES Flow

This Level 3 Data Flow Diagram provides a detailed view of the complete transactional flow within the Payment Enrollment System, illustrating how various actors interact with the system in a real-world sequence. It captures key operational steps, from HR personnel onboarding employees and granting system access, to employees logging attendance, submitting leave requests, and accessing payslips. The Finance Team initiates payroll processing based on stored employee data, while the Payment System handles calculations, disbursement, and communication with the database for fund transfers.

The diagram also accounts for exception handling, such as payment failures, audit trail generation, and administrator involvement in investigating system-level issues. Each message and data exchange is mapped clearly across lifelines, emphasizing timing, flow of control, and system responsibility. This level of decomposition is valuable for visualizing dynamic interactions, validating process logic, and supporting technical implementation and troubleshooting efforts.



UML CLASS DIAGRAM

Constantinos.

The diagram below illustrates the main classes involved in the Payment Enrollment System, along with their attributes, operations, and relationships. It captures core entities such as employees, HR staff, the finance department, and the bank system, as well as supporting classes like payslips, leave requests, and payroll records. Each class defines its structure and behavior, while the arrows indicate how these entities are connected within the system.

This class diagram is instrumental in the design and development phases of the project. It defines the system's object-oriented structure, ensuring consistency between the software architecture and the underlying business logic. Developers can use it to implement class-based components, while database designers can derive table schemas from the attributes. Relationships such as "submits," "includes," and "sends to" guide interaction logic between modules. For example, how leave requests are submitted by employees and processed by HR. Additionally, the operations defined in each class (like `approveDisbursement()` or `generatePDF()`) help outline key system functionalities to be implemented in code.

The interactions shown in the diagram reflect how different components collaborate in the system. For example, HR staff onboard employees and review leave requests, which are initiated by employees. The finance department interacts with payroll records to review and approve disbursements, which are then processed by the bank system. Supporting components like `LeaveRequest` and `Payslip` are tightly coupled with the `Employee` class, showing direct submission and generation behaviors. These interactions represent the core operational logic of the system.

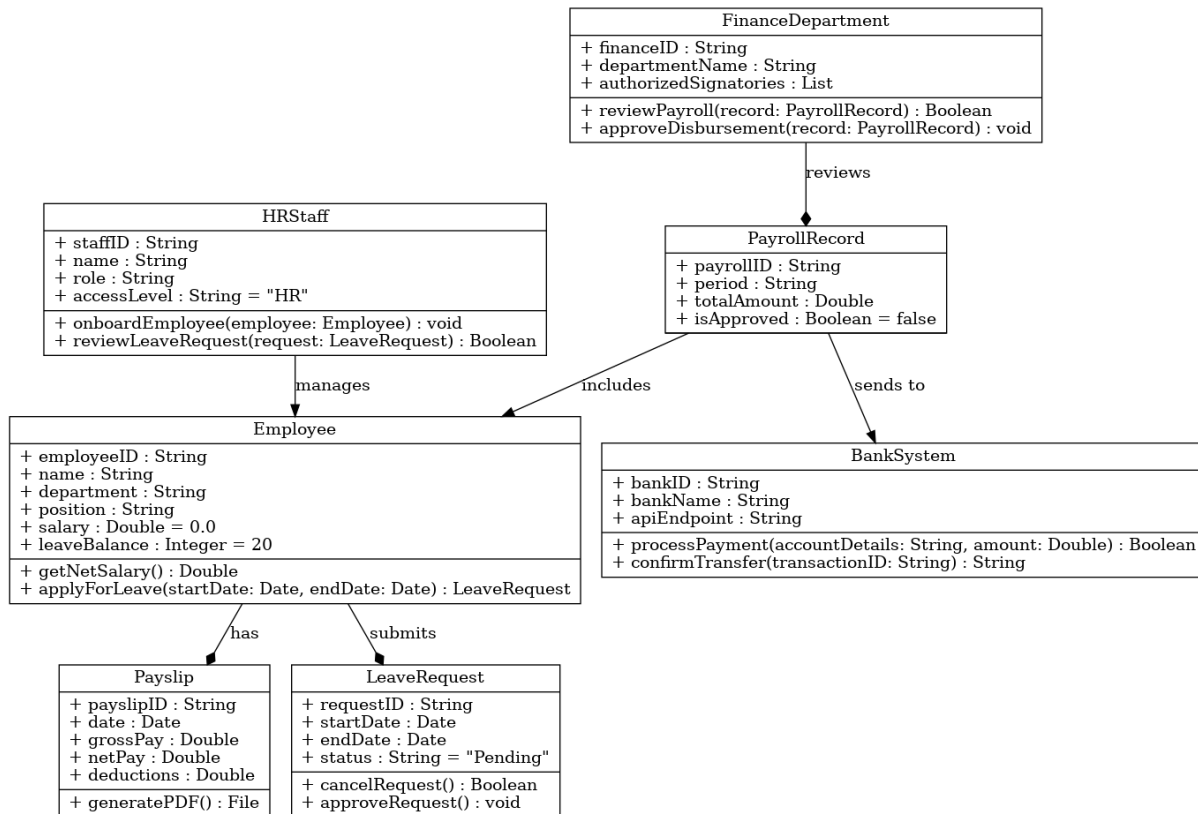


Figure 5: UML Class Diagram for PES

SEQUENCE DIAGRAM

Constantinos

Sequence diagrams represent the chronological flow of interactions between objects and actors in the context of a specific functionality or use case. They focus on the exchange of messages over time and provide a detailed view of how behavior is distributed across system components.

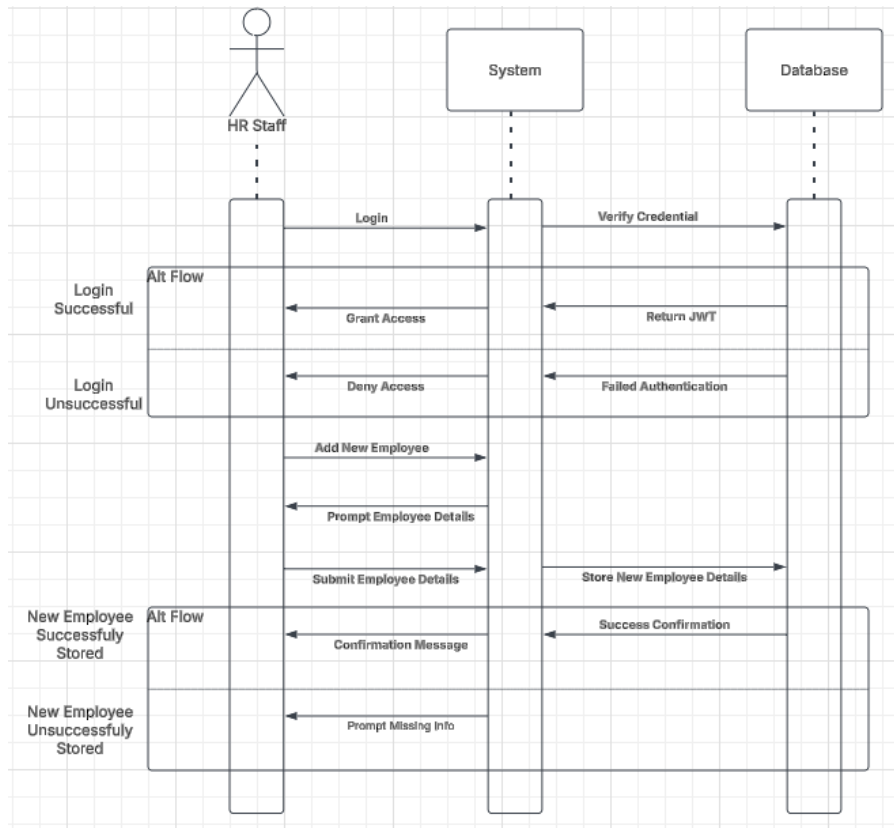
- **Clarify logic and order:** They explicitly show the **chronological order** of interactions, helping stakeholders understand how operations are coordinated.
- **Bridge between requirements and design:** They serve as a visual aid that connects **use cases** to **implementation logic**, making it easier for developers to translate behavior into code.
- **Assist in identifying responsibilities:** By revealing which object handles which task, they support responsibility assignment during design.
- **Support communication:** They provide a **clear, visual representation** that can be understood by technical and non-technical stakeholders alike.

- **Aid debugging and testing:** By documenting expected interaction flows, they serve as a reference for verifying correct system behavior.

Sequence Diagram 1: Employee Onboarding

This sequence diagram illustrates the interactions that occur during the **Employee Onboarding** process within the Payment Enrollment System. The primary participants involved are the **HR Personnel** and the **System**. The diagram captures the communication flow between these entities, clearly delineating their respective responsibilities. It emphasizes the system's role in validating input, coordinating actions, and securely managing employee data during the onboarding procedure.

1. **Initiation by HR Personnel:** The sequence begins when the HR Personnel initiates the onboarding process by submitting a request to the system.
2. **Data Validation and Record Creation:** The system responds by validating the provided employee data and then interacts with the Employee Record Module to create a new employee entry.
3. **Confirmation and Acknowledgment:** Once the record is successfully created, the system sends a confirmation message back to the HR Personnel.



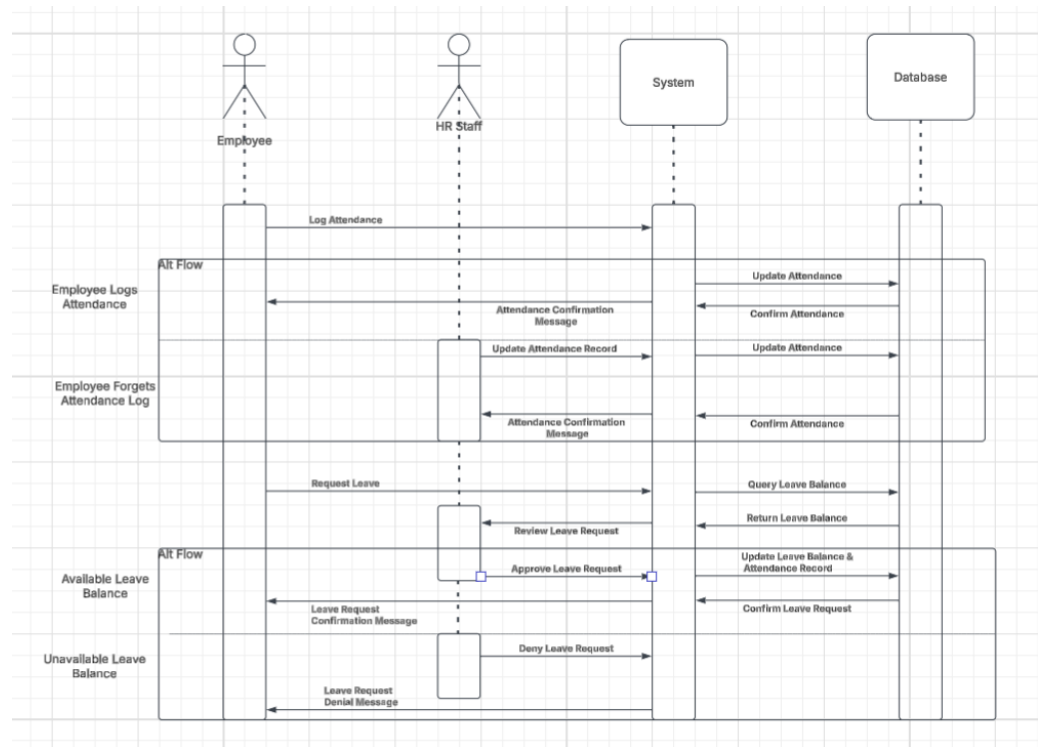
UC1: Employee Onboarding

Sequence Diagram 2: Attendance & Leave Management

This sequence diagram depicts the interactions for the **Attendance Logging and Leave Request** process within the Payment Enrollment System. The involved actors are the **Employee, HR Staff, System, and Database**. It outlines two primary flows: attendance logging (including alternate handling when an employee forgets to log attendance) and the process of requesting and approving leave.

The diagram showcases how attendance data is validated, updated, and confirmed by the system, with intervention from HR Staff in correction scenarios. Additionally, it visualizes the procedure for handling leave requests—beginning with submission, proceeding through HR review, and concluding with approval or denial based on available leave balance, which is verified and updated in the database.

Overall, the sequence diagram highlights the system's role in ensuring accurate attendance tracking and leave management while supporting user interaction and administrative validation.



UC2: Attendance & Leave Management

CONTROL FLOW DIAGRAM

Constantinos.

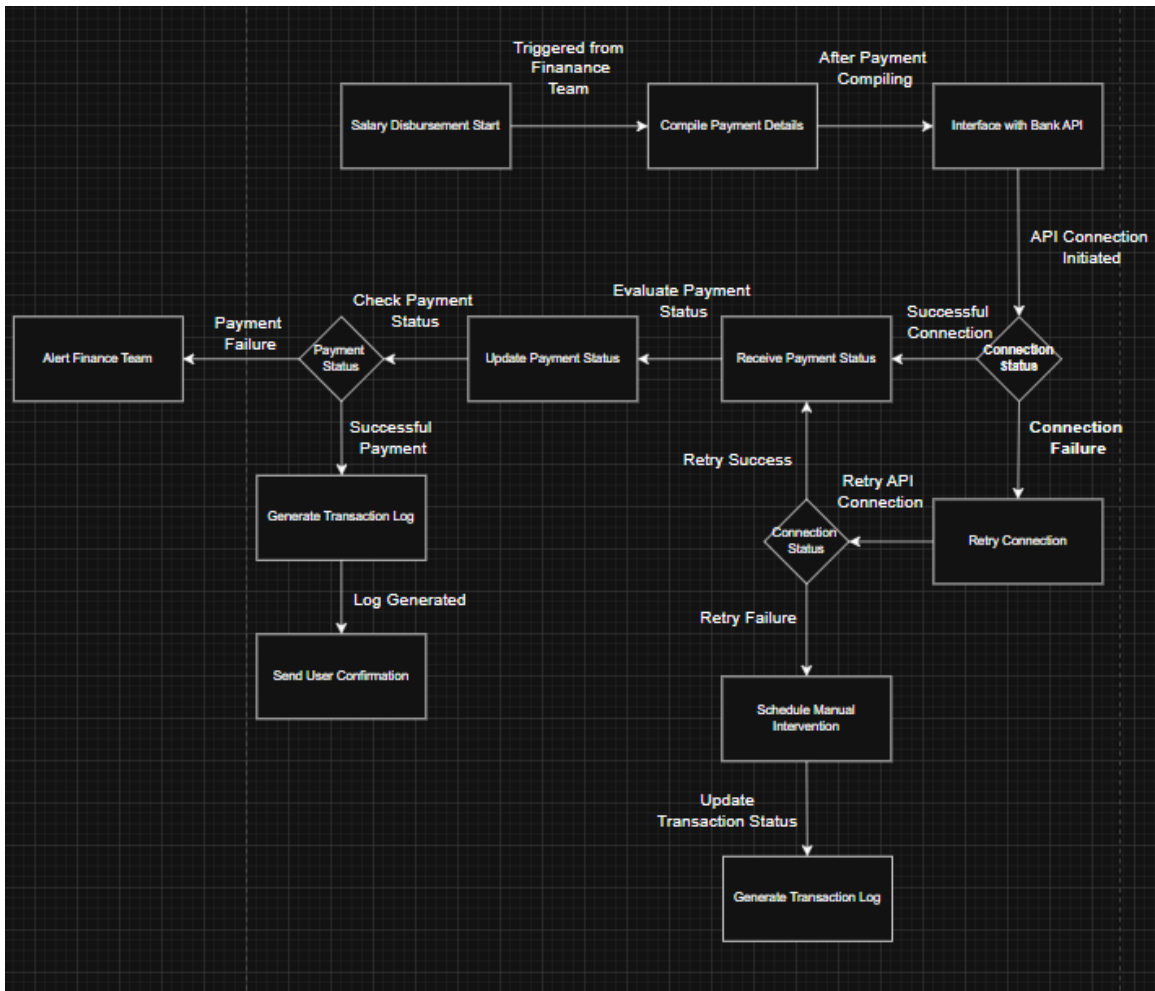
Control Flow Diagrams (CFDs) are used to model the sequence, branching, and decision-making logic of system operations. They are particularly useful in visualizing **business workflows**, **error-handling scenarios**, and **conditional execution paths**. CFDs help translate use case descriptions into a structured visual representation that can support both **technical implementation** and **stakeholder communication**.

CFDs are rooted in principles from **structured analysis and design**, where systems are broken down into discrete, well-defined actions. They also incorporate concepts from **behavioral modeling** and **fault-tolerant design**, allowing developers to account for alternative and exception flows, such as retries, failures, and manual overrides.

Control Flow Diagram 1: Payment Disbursement

This Control Flow Diagram illustrates **Use Case 4: Payment Disbursement** in the Payment Enrollment System. It outlines the process from initiation to completion, including both successful and failure handling.

1. **The finance team starts the process**, triggering the system to compile payment details and send them to the **Database**.
2. A **connection status check** determines the next step:
 - a. If successful, the system proceeds to **receive and evaluate the payment status**.
 - b. If it fails, the system attempts a **retry**.
3. On **retry failure**, the system schedules **manual intervention** and logs the issue. If the retry is successful, it continues to payment evaluation.
4. Once the **payment status is received**, the system:
 - a. Updates the payment status,
 - b. Generates a transaction log, and
 - c. Sends a confirmation if payment is successful.
5. If the **payment fails despite a valid connection**, the system **alerts the finance team**.



CFD 1: Payment Disbursement

Control Flow Diagram 2: Compliance & Financial Reporting

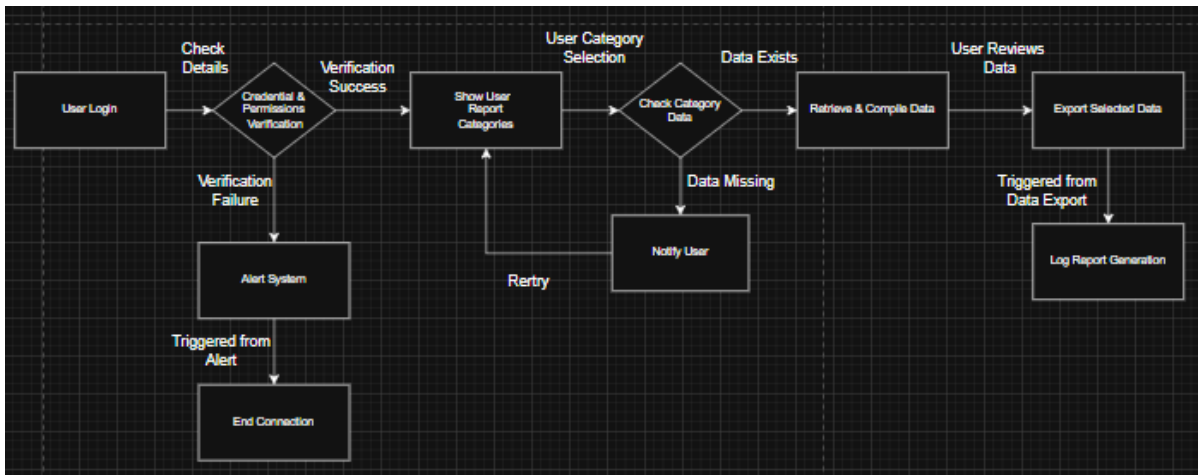
This Control Flow Diagram illustrates **Use Case 6: Compliance and Financial Reporting** within the Payment Enrollment System. It outlines the flow from user authentication to report export and logging, including error handling and retries.

1. The process begins when a **finance team member logs in**.
2. The system performs **credential and permission verification**.
 - a. If verification fails, the system **alerts security** and ends the session.
3. If access is granted, the user is shown **available report categories**.
4. Upon **selecting a category**, the system checks for associated data:
 - a. If data is available, it is **retrieved and compiled** for review.

- b. If data is missing, the system **notifies the user**, who can **retry** and make another selection.

5. Once data is reviewed, the user can **export the selected report**, and the system **logs the export action** for auditing purposes.

This CFD ensures secure access, guides users through conditional flows, and maintains full traceability of report activity, making it suitable for regulated environments.



CFD 2: Compliance & Financial Reporting

TEST PLAN

A test plan is a crucial part when designing a software system because everything built up to this point is being put to the test in order to make sure that the functional parts of the system work as they're intended to. Testing the system also saves a lot of time and effort as well as money since any wrongdoings or misbehaviours of the system's functions are detected and repaired before deployment.

The first step in testing the system's functions is to determine what should be tested by going through the system's requirements and its use cases. This is the best way since it does not only help decide what functions should be put to the test but it also makes sure that the team works together and communicates with each other in order to find the best course of action.

The next step, after we determine what is to be tested, is to write the test plan as follows:

- For each test case we should explain what the module should be doing and how the system must respond when the module is used.
- Select what inputs should be tested for each case.
- Explain how we came up with the selected inputs.
- Determine what the expected output is for every input.
- Decide what the success criteria is and what is a pass or a fail.
- What the pre-conditions are in order for the module to be functional.

TEST CASES

TEST CASE 1 (TC01):

Module Description: The “Employee login” module is the first step for an employee to log in to their account on the PES system in order to review their salary deductions, tax calculations etc. The system must authenticate that their email and account are valid and have been approved by the company in order to prevent any data leaks as well as to only reveal the information which the employees access allows them.

Module Description: The “Login” module is the first step for an employee to log in to their account on the PES system in order to review their salary deductions, tax calculations etc. The system must authenticate that their email and account are valid and have been approved by the company in order to prevent any data leaks as well as to only reveal the information which the employees access allows them.

Selected Inputs:

1. **Employee email:** The employee’s email with which the system remembers them.
2. **Employee Password:** Ensures that the corresponding password is correct and allows employees to log in.
3. **Invalid Inputs:** Test cases for which the password was incorrect, fields have been left blank or the employee selected the wrong role to describe themselves (HR instead of Employee etc).

Methodology for Input Selection: Inputs were chosen based on system requirements and different user scenarios with different access levels (CO, HR etc.) to ensure that not all employees can log in and access the same files.

Expected Outputs:

1. **Employee email:**
 - The system should identify whether the employee email is valid.
 - Once the employee email is entered validly, the system should confirm by turning green.
2. **Employee Password:**
 - The system should proceed to the next page once the password is entered successfully.
 - If the password is incorrect, a message should pop up notifying the user that they have “x” attempts available.
3. **Invalid Inputs:**
 - Display appropriate error messages telling the employee what went wrong and guiding them to correct entries.

Success Criteria:

- All inputs are processed without errors.
- Successful login with appropriate access level.
- The system correctly processes both valid and invalid inputs.

Pre-Conditions:

1. The user is an employee with a company account.
2. The employee has completed the probation period and can login.

Edited by Tommy

TEST CASE 2 (TC02):

Module Description: The “salary calculation” module will take into account all salary deductions (tax, health insurance etc.), as well as any vacation days the employee has taken and calculate the monthly salary for the employee.

Selected Inputs:

1. **Part-Time Employee:** Calculating the salary of an employee who gets paid by hours worked.
2. **Full-Time Employee:** Calculating the salary of an employee who gets paid monthly after all deductions.
3. **Salary Calculation with Optional Fields:** An employee can calculate what their salary would be next month by inputting more information (vacation days, overtime etc.).
4. **Invalid Inputs:** Test cases involving invalid inputs such as blank required fields, negative quantities etc. as well as for employees who have not completed their probationary period yet.

Methodology for Input Selection: Inputs were chosen based on system requirements and potential user scenarios to ensure coverage of both normal operations and edge cases.

Expected Outputs:

1. **Part-Time Employee:**
 - Show the employee how many hours they worked this month.
 - Show the calculations made and the final salary.
2. **Full-Time Employee:**
 - Show the employee all deductions/additions for the current month.
 - Show the employee their salary after all deductions/additions.
3. **Salary Calculation with Optional Fields:**
 - Show the employee next month's salary and show the optional fields the employee can fill to further calculate their salary.

4. Invalid Inputs:

- Display appropriate error messages.

Success Criteria:

- All calculations are made correctly for all cases.
- The system handles all valid and invalid inputs correctly.

Pre-Conditions:

1. The employee is logged in to their account.
2. The employee's status is not pending.

TEST CASE 3 (TC03):

Module Description: The “payslip generation” module will take into account all data regarding the employee as well as all the data for the employee's salary and it will generate their monthly payslip upon request.

Selected Inputs:

1. **Part-Time Employee:** Generate a payslip for a part-time employee and show information available only to part-time employees in general (hours worked, hourly wage etc.).
2. **Full-Time Employee:** Generate a payslip for a full-time employee and show information available only to full-time employees in general (health insurance deductions, overtime additions etc.).
3. **Invalid Inputs:** Test cases involving invalid inputs for employees who have not finished their probation periods yet or have pending statuses.

Methodology for Input Selection: Inputs were chosen based on system requirements and potential user scenarios.

Expected Outputs:

1. **Part-Time Employee:**
 - The payslip is generated.
 - The payslip contains all the information a part-time employee's payslip should have.
2. **Full-Time Employee:**
 - The payslip is generated.
 - The payslip contains all the information a full-time employee's payslip should have.
3. **Invalid Inputs:**

- Display appropriate error messages.

Success Criteria:

- The payslips are generated each time and are correct.
- The system handles all valid and invalid inputs correctly.

Pre-Conditions:

1. The employee is logged in to their account.
2. The employee's status is not pending.

TEST CASE 4 (TC04): LEAVE REQUEST AND APPROVAL SYSTEM**Module Description:**

The "Leave Management" module allows employees to submit leave requests and enables HR/management to approve or deny them. The system checks for leave balance, overlaps in requests between two employees who cannot be absent at the same time and notifies the necessary parties.

Selected Inputs:

- **Valid Leave Request:** Employee with available leave submits a request.
- **Overlapping Leave Request:** Leave request overlaps with another employee's approved leave.
- **Exceeded Leave Quota:** Employee requests more days than allowed.
- **Invalid Inputs:** Dates left blank, start date after end date, request during probation.

Methodology for Input Selection: Inputs were chosen based on system requirements and potential user scenarios. We also take into account different user behaviors or access levels to the system.

Expected Outputs:

1. **Valid Request:**
 - The employee makes their request.
 - The leave request is under review within the system.
2. **Detected Overlap:**
 - System flags potential conflict to the user.
 - The employee submits their request with some comments explaining the importance of the leave request.
 - The leave request is under review within the system.
3. **Exceeded Quota:**
 - The system notifies the employee that they exceeded the limit.

- The employee submits their request with some comments explaining the importance of the leave request.
- The leave request is under review within the system.
- The employee is notified that their leave would be unpaid.

4. **Invalid Inputs:**

- Display appropriate error messages.

Success Criteria:

- All valid requests go through.
- All conflicts and policy violations are flagged properly.
- Invalid requests are rejected with appropriate feedback.

Pre-Conditions:

1. The employee is logged in to their account.
2. Attendance and leave records are up to date.

TEST CASE 5 (TC05): EMAIL NOTIFICATION SYSTEM

Module Description:

The “email notification system” module is responsible for informing the employee for any pending requests or new notifications that come through the system. Their account is linked to their email address and any new information that the employee must be notified about will be sent to their email without any need to log in to the system.

Selected Inputs:

1. **Leave Request Handled:** When a leave request is handled.
2. **Leave Pending:** A leave request is still pending after one week.
3. **Payslip Generated:** The employee requested their payslip.
4. **Invalid Inputs:** The email address linked to the account has a full inbox or cannot receive emails.

Methodology for Input Selection: Inputs were chosen based on system requirements and potential user scenarios.

Expected Outputs:

1. **Leave Request Handled:**

- The employee receives an email informing them about their leave request.

2. **Leave Pending:**

- The department responsible for handling leave requests are notified through email.

3. **Payslip Generated:**

- The employee receives the payslip on their email address.

4. **Invalid Inputs:**

- Display appropriate error messages.

Success Criteria:

- Notifications arrive at the correct email addresses.
- Errors handled without crashing the system.

Pre-Conditions:

1. Email addresses are properly configured in employee records.
2. Employees email inbox are not full.

Tommy

TRACEABILITY OF REQUIREMENTS WITH USE CASES AND TEST CASES

Another important part to the test planning is to build a table showing that the test cases have the system's requirements covered. This means that the requirements are connected to a use case as well as a test case, explaining in brief what the system requirements are and how they will be tested. The table acts as a small "map" that shows how the requirements are connected to a use case and also how it will be tested.

The following table shows how the test plan is connected to the system's requirements and its use cases:

Table 3: Requirements Traceability

Requirement ID	Description	Use Case	Test Case
RQ01	Employee Onboarding - Creating an account for each employee	UC01	TC01, TC05
RQ02	Salary Calculation	UC03, UC04	TC02, TC05
RQ03	Leave Management	UC02	TC04, TC05
RQ04	Payslip Generation	UC05, UC06	TC03, TC05

By Tommy

PROFESSIONAL STANDARDS

Section 7: Professional Standards, by Peter M 

To ensure reliability, security, and compliance, the Payment Enrollment System adheres to the following industry standards:

- **GDPR (General Data Protection Regulation):** Ensures employee salary and personal data are securely stored and accessed only by authorized personnel. The GDPR will be used with the database so that the data is secure and will not be accessed by anyone in general except the parties that are authorised to do so.
 - Employee data must be encrypted at rest and during transit(that meaning, whilst it's stored in the database, and while it's being used in the logic layer).
 - Data from users that are no longer in the system(ie. unemployed) needs to have a lifespan before it's discarded by the system.
 - With User Authentication, role-based access is implemented to restrict data access.
 - The modules affected by this are **everything in the Data Layer**, as well as the **Auth Service, Payroll Service** and **Reporting Service** from the Logic Layer.
- **RESTful API Best Practices:** Ensures efficient communication between system components for seamless data exchange. The API will be used between the system and the bank systems so that they work coherently between themselves and work appropriately so that the payments will be done correctly.
 - Backend modules that need to communicate with outside APIs are affected: **Payroll, Auth Service, Notifications, Bank API** modules.

- **Financial Compliance Regulations:** Maintains accurate financial records to meet auditing and legal requirements. This will be integrated in the database and backend of the system so that when the payslips are generated to be audited the financial department will be able to identify deductions made by the system.
 - Automated tax/GESY calculations aligned with national laws.
 - Transaction logs for audits must be immutable (and linearized if the system is scaled to multiple databases).
 - The components affected are the **Payroll Service**, the **Salary Calculation** part of the business logic layer, and the **Reporting Service**. On top of this, the **Data Layer** itself needs to be constructed in such a way so as to follow the country's payment standards

MAPPING TABLE

Standard	System Component (	Key Implementation
GDPR	EmployeeDB, PayrollDB, LeaveDB	Encryption during rest & transit. Data lifetime, role-based access
Financial Compliance	Payroll Module, Reporting Service, Payslip Service, EmployeeDB, LeaveDB, PayrollDB	Automated tax calculations, NoSQL database that can easily adapt to other banks' DB schemas.

By following these professional standards, the PES guarantees security, compliance, and operational efficiency, making it a robust payroll management solution.

Section 7 End

PAGE INTENTIONALLY LEFT BLANK.

SEMESTER 2 (FALL 2025-6)

PROJECT PROGRESS REPORT

By Chrysostomos Demosthenous, Andreas Karpasitis, Peter Maravgakis, Ireneos Pavlou, Constantinos Stephanides

Abstract: This document is a follow-up technical supplement to the main project report that **Group 2** completed in the Spring 2025 semester and reading it is recommended. In this document we will break down the system's high-level use cases into specific submodules and pseudocode, establish the responsibility of each member, provide a traceable mapping for requirements, and outline our implementation schedule. The members of **Group 2** as well as their GitHub profiles are as follows:

[Click Here to access our Github Repository](#)

Member No.	Name	GitHub Profile
M1	C. Demosthenous	tomdem1695
M2	A. Karpasitis	Karpas15
M3	P. Maravgakis	Bobabytes
M4	I. Pavlou	eirinaiospavlou
M5	C. Stephanides	Thkiavre

FUNCTIONAL REQUIREMENTS

Requirement ID	Modules/Use Cases	Person assigned	
FR-00A	Routing System	M3	P. Maravgakis
FR-00B	Database Schema	M1	C. Demosthenous
FR-00C	Frontend	M3	P. Maravgakis
FR-01	Login User	M3	P. Maravgakis
FR-02	Employee Onboarding/Offboarding	M2	A. Karpasitis
FR-02b	Update Employee personal data	M2	A.Karpasitis
FR-03	Attendance Recording	M5	C. Stephanides
FR-04	Leave Request	M2	A. Karpasitis
FR-05	Leave Management	M2	A. Karpasitis
FR-06	Payroll Processing	M1	C. Demosthenous
FR-07	Payment Disbursement	M1	C. Demosthenous
FR-08	Viewing and Downloading	M5	C. Stephanides
FR-09	Financial Reporting	M3	P. Maravgakis
FR-10	Update Salary	M5	C. Stephanides
FR-11	User Authentication	M2	A. Karpasitis
FR-12	Portals	M3	P. Maravgakis

SYSTEM DESIGN

Database	Backend	Frontend
SQLite	Node.js	React

As discussed in the previous document and semester, PES will initially be deployed as a web application utilizing the client-server model, therefore these are the systems we will use on each architectural category.

PROGRESS REPORT DOCUMENT CHANGELOG

Version	Date	Details	By	
2.0 (FINAL)	20/Dec/2025	Final release	M1, M2, M3, M5	P. Maravgakis, C. Stephanides, C. Demosthenous, A. Karpasitis
1.8	17/Dec/2025	- Updated Gantt Chart - Updated Test Cases - Updated User Manual	M3, M5, M2	P. Maravgakis, C. Stephanides, A. Karpasitis
1.7	10/Dec/2025	- Added user manual - Updated Gantt Chart	M3, M2	P. Maravgakis, C. Stephanides,
1.6	02/Dec/2025	- Test Cases: TC-08A, TC-08B, TC-12E, TC-12H, TC-12F, TC-12A - Updated Gantt Chart	M3	P. Maravgakis
1.5	25/Nov/2025	- More Test Cases for Leave submodule - Updated Gantt Chart	M1, M2, M3, M5	P. Maravgakis, C. Stephanides, C. Demosthenous, A. Karpasitis
1.4	19/Nov/2025	- Introduced Test Cases - Updated Gantt Chart	M3, M5	P. Maravgakis, C. Stephanides,
1.3	11/Nov/2025	- Added Gantt Chart - Updated & Expanded Gantt Chart with Submodules(The standard for us will now be Frontend(w/ indents for submodules), Backend(w/ indents for submodules), Database(w/ indents for submodules)	M2, M3	P. Maravgakis, A. Karpasitis
1.2	06/Nov/2025	- Drew & added sketches - Formalized Requirements & Members lists - Redesigned document - Rewrote pseudocode for FR-09	M3	P. Maravgakis
1.1	29/Oct/2025	Deadline, Initial Document Publication	All	Everyone
1.0	22/Oct/2025	Initial Document Creation	All	Everyone

GANTT CHART (ACCESSIBLE HERE)

V1.7: Collaborated by everyone, LAST UPDATED 21/DEC/2025.

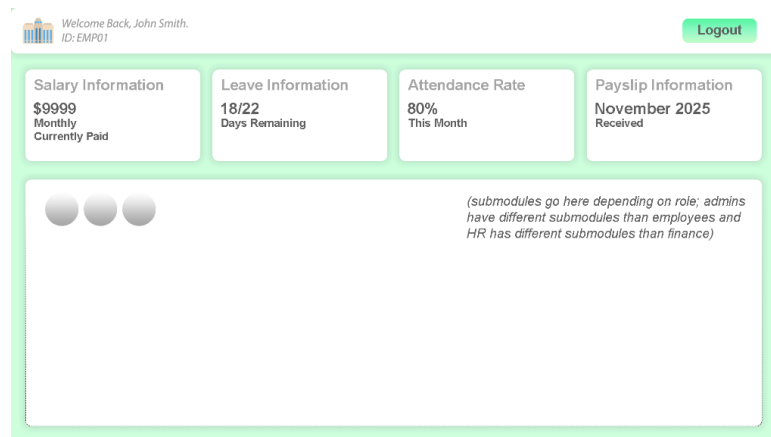
We've chosen to include the Gantt Chart as a separate .xlsx file alongside our submission. If you'd like the online link, click the header above.

PAGE INTENTIONALLY LEFT BLANK.

SYSTEM MODULE PSEUDOCODE

This section breaks down the high-level system into more specific pseudocode for the purpose of efficiently distributing the task load of development between the users.

FR-00A: MAIN ROUTING SYSTEM



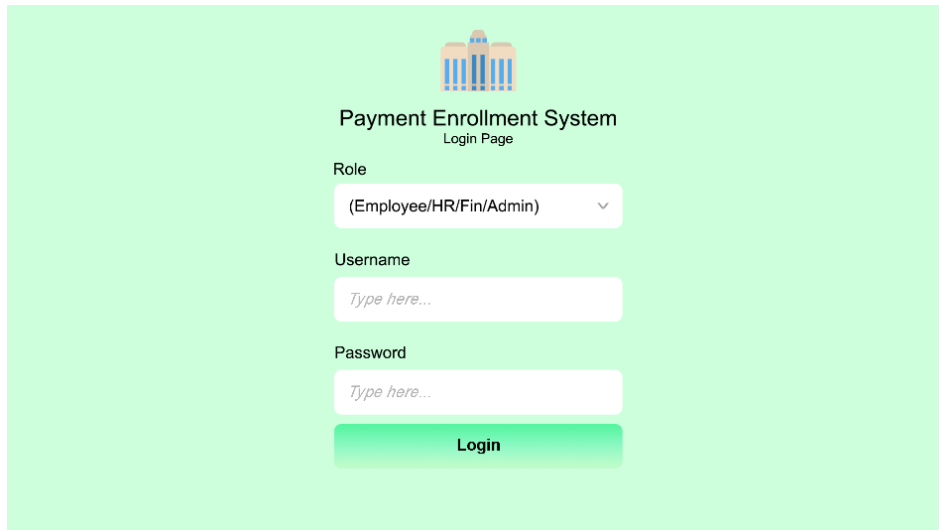
Function: MAIN_APPLICATION_LOOP()

Description: Initializes the system and routes users to the appropriate portal based on the authentication state.

FR: FR-00

```
PROGRAM Main_Application_Loop()  
  
    DATABASE employees_db  
  
    DATABASE attendance_db  
  
    DATABASE leave_db  
  
    DATABASE payroll_db  
  
    WHILE system_running DO  
        user_type = LOGIN_FUNCTION()  
  
        CASE user_type OF  
            "hr": HR_MENU()  
            "manager": MANAGER_MENU()  
            "employee": EMPLOYEE_MENU()  
            "financial": FINANCIAL_MENU()  
            Default: NOT_FOUND_PAGE()  
        END CASE  
    END WHILE  
  
END PROGRAM
```


FR-01: MAIN LOGIN FUNCTION



The image shows a login page for a system titled "Payment Enrollment System". At the top center is a small icon of a building with a clock tower. Below the icon, the title "Payment Enrollment System" is displayed in a bold, black font, with "Login Page" in a smaller font directly underneath. The form consists of several elements: a "Role" label followed by a dropdown menu showing "(Employee/HR/Fin/Admin)" with a downward arrow; a "Username" label followed by a text input field containing the placeholder "Type here..."; a "Password" label followed by another text input field with the same placeholder; and a green "Login" button at the bottom.

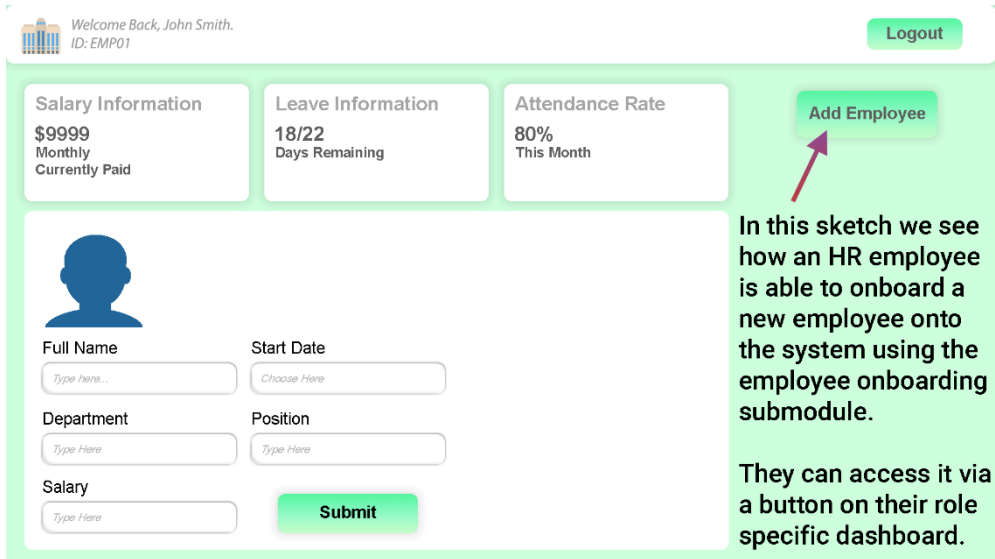
Function: LOGIN_MODULE()

Description: Handles User Authentication.

FR: FR-01

```
FUNCTION LOGIN_FUNCTION()  
  
    DISPLAY "Please enter your credentials"  
  
    INPUT username  
  
    INPUT password  
  
  
    QUERY database: SELECT * FROM users WHERE username = username AND  
password = password  
  
  
    IF credentials_found THEN  
  
        user_role = GET_USER_ROLE(username)  
  
        DISPLAY "Login successful"  
  
        RETURN user_role  
  
    ELSE  
  
        DISPLAY "Invalid credentials. Please try again."  
  
        RETURN LOGIN_FUNCTION() // Recursive call for retry  
  
    END IF  
  
END FUNCTION
```

FR-02: EMPLOYEE ONBOARDING FUNCTION:



The sketch shows a user interface for an HR employee's dashboard. At the top, it says "Welcome Back, John Smith. ID: EMP01" and has a "Logout" button. Below this are three summary cards: "Salary Information" showing "\$9999 Monthly Currently Paid", "Leave Information" showing "18/22 Days Remaining", and "Attendance Rate" showing "80% This Month". To the right of these cards is a green "Add Employee" button with a red arrow pointing to it. Below the cards is a form for adding a new employee, featuring a blue silhouette icon and input fields for "Full Name", "Start Date", "Department", "Position", and "Salary", along with a green "Submit" button.

Welcome Back, John Smith.
ID: EMP01

Logout

Salary Information
\$9999
Monthly
Currently Paid

Leave Information
18/22
Days Remaining

Attendance Rate
80%
This Month

Add Employee

In this sketch we see how an HR employee is able to onboard a new employee onto the system using the employee onboarding submodule.

They can access it via a button on their role specific dashboard.

Full Name
Type Here...

Start Date
Choose Here

Department
Type Here

Position
Type Here

Salary
Type Here

Submit

Function: EMPLOYEE_ONBOARDING()

Description: Registers a new employee in the system. This function is specific to the HR Employee class.

FR: FR-02

```
FUNCTION EMPLOYEE_ONBOARDING()  
  
    DISPLAY "New Employee Onboarding Form"  
  
    INPUT employee_data: full_name, start_date, start_salary,  
department, position  
  
    // Generate unique ID and credentials  
  
    employee_id = GENERATE_UNIQUE_ID("EMP")  
  
    email = GENERATE_EMAIL(full_name)  
  
    temp_password = GENERATE_RANDOM_PASSWORD()  
  
    // Save to databases  
  
    EXECUTE_SQL("INSERT INTO employees_db (id, full_name, start_date,  
salary, department, position) VALUES (?, ?, ?, ?, ?, ?)", employee_id,  
full_name, start_date, start_salary, department, position)  
  
    EXECUTE_SQL("INSERT INTO users_db (username, password, role,  
employee_id) VALUES (?, HASH(?), 'employee', ?)", email, temp_password,  
employee_id)  
  
    DISPLAY "Employee " + full_name + " onboarded successfully."  
  
    DISPLAY "Credentials - Email: " + email + " | Temp Password: " +  
temp_password  
  
END FUNCTION
```

FR-03: RECORD ATTENDANCE FUNCTION:

Welcome Back, John Smith.
ID: EMP01

Logout

Salary Information
\$9999
Monthly
Currently Paid

Leave Information
18/22
Days Remaining

Attendance Rate
80%
This Month

Attendance

Choose Workday

January 2015

Su	Mo	Tu	We	Th	Fr	Sa
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Monday, January 26

Person	Hours	Present?
EmpID:Fullname xx:yy-zz:aa		X ✓
EmpID:Fullname xx:yy-zz:aa		X ✓
EmpID:Fullname xx:yy-zz:aa		X ✓
EmpID:Fullname xx:yy-zz:aa		X ✓

Submit

In this sketch we see how a manager is able to record the attendance of their team members for a particular day using the record_attendance submodule.

It's accessible via a button on their role specific dashboard.

Function: RECORD_ATTENDANCE()

Description: Allows a manager to record the attendance of their team members.

FR: FR-03

```
FUNCTION RECORD_ATTENDANCE()

    DISPLAY calendar_view

    SELECT employee_from_list

    SELECT date_from_calendar

    INPUT attendance_status:

        OPTIONS: "present", "absent", "late", "sick_leave"

    IF attendance_status = "absent" THEN

        INPUT absence_reason

        IF absence_reason = "none" THEN

            UPDATE attendance_db SET status = "unauthorized_absence"

            RECALCULATE_SALARY(employee_id) // Deduct for unauthorized
absence

        END IF

    END IF

    SAVE_TO_DATABASE(employee_id, date, attendance_status)

    UPDATE_CALENDAR_DISPLAY()

END FUNCTION
```

FR-04: LEAVE REQUEST FUNCTION:

Welcome Back, John Smith.
ID: EMP01

Logout

Salary Information
\$9999
Monthly
Currently Paid

Leave Information
18/22
Days Remaining

Attendance Rate
80%
This Month

Leave Request

Start Date
January 2015
Su Mo Tu We Th Fr Sa
28 29 30 31 1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Monday, January 26

End Date
January 2015
Su Mo Tu We Th Fr Sa
28 29 30 31 1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Monday, January 26

Type
Sick Leave/Annual Leave

Notes
Type here...

Submit

This is how an employee of the company requests leave from work using the leave request submodule.

Function: SUBMIT_LEAVE_REQUEST(employee_id)

Description: Allows an employee to submit a leave request.

FR: FR-04

```
FUNCTION LEAVE_REQUEST(employee_id)

    DISPLAY "Leave Request Form"

    INPUT leave_type // "annual" or "sick"

    INPUT start_date

    INPUT end_date

    days_requested = CALCULATE_DAYS(start_date, end_date)

    available_leaves = GET_AVAILABLE_LEAVES(employee_id)

    IF leave_type = "annual" THEN

        IF days_requested > available_leaves THEN

            DISPLAY "Error: Not enough annual leaves available"

            RETURN

        END IF

    ELSE IF leave_type = "sick" THEN

        UPLOAD doctor_note

        IF doctor_note_not_provided THEN

            DISPLAY "Error: Doctor's note required for sick leave"

            RETURN

        END IF

    END IF

    INSERT INTO leave_requests_db VALUES (employee_id, leave_type, start_date, end_date, "pending")

    DISPLAY "Leave request submitted successfully"

END FUNCTION
```

FR-05: LEAVE MANAGE FUNCTION:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: MANAGE_LEAVES()

Description: Allows HR or Managers to view, approve, or decline pending leave requests.

FR: FR-05

```
FUNCTION MANAGE_LEAVES ()

    pending_requests = QUERY "SELECT * FROM leave_requests_db WHERE status =
'pending'"

    FOR EACH request IN pending_requests DO

        DISPLAY request_details:

            employee_name

            leave_type

            dates

            duration

        // Check for date collisions

        IF CHECK_DATE_COLLISION(request) THEN

            DISPLAY "Warning: Date collision detected"

        END IF

        DISPLAY "Approve or Decline?"

        INPUT decision

        IF decision = "approve" THEN

            UPDATE leave_requests_db SET status = "approved"

            NOTIFY_HR_FOR_RECORDING(request)

        ELSE

            UPDATE leave_requests_db SET status = "declined"

            NOTIFY_EMPLOYEE(request.employee_id, "Leave declined")

        END IF

    END FOR

END FUNCTION
```

FR-06: PAYROLL PROCESSING FUNCTION:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: PROCESS_PAYROLL()

Description: Calculates net pay for employees based on attendance, leaves, and deductions.

FR: FR-06

```
FUNCTION PROCESS_PAYROLL()

    DISPLAY "Select Employee for Payroll"

    INPUT employee_id

    // Fetch employee data

    employee_data = QUERY "SELECT * FROM employees_db WHERE id = employee_id"

    attendance_data = QUERY "SELECT * FROM attendance_db WHERE employee_id = employee_id
AND month = current_month"

    leave_data = QUERY "SELECT * FROM leave_db WHERE employee_id = employee_id AND month
= current_month"

    // Calculate salary components

    base_salary = employee_data.salary

    deductions = CALCULATE_DEDUCTIONS(attendance_data, leave_data)

    gross_salary = base_salary

    net_pay = gross_salary - deductions

    // Display for verification

    DISPLAY "Payroll Summary:"

    DISPLAY "Base Salary: " + base_salary

    DISPLAY "Deductions: " + deductions

    DISPLAY "Net Pay: " + net_pay

    DISPLAY "Confirm payroll calculation? (yes/no)"

    INPUT confirmation

    IF confirmation = "yes" THEN

        INSERT INTO payroll_db VALUES (employee_id, current_month, gross_salary,
deductions, net_pay, "processed")

        DISPLAY "Payroll processed successfully"

    ELSE

        DISPLAY "Payroll processing cancelled"

    END IF

END FUNCTION
```

FR-07: PAYMENT DISBURSEMENT FUNCTION:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: SALARY_DISBURSEMENT()

Description: Processes approved payroll by transferring funds via banking API. For the purposes of this course we don't have a banking API on hand: however, we will code the boilerplate function that a banking API would be routed into. We assume that the banking API has its own security measures.

FR: FR-07

```
FUNCTION SALARY_DISBURSEMENT()

    pending_payments = QUERY "SELECT * FROM payroll_db WHERE status = 'processed' AND payment_date =
current_date"

    FOR EACH payment IN pending_payments DO

        DISPLAY "Processing payment for: " + payment.employee_name

        DISPLAY "Amount: " + payment.net_pay

        // Interface with payment system

        payment_result = PROCESS_PAYMENT(payment.employee_bank_details, payment.net_pay)

        IF payment_result = "success" THEN

            UPDATE payroll_db SET status = "paid", payment_date = CURRENT_DATE

            CREATE_LOG("Payment successful for " + payment.employee_id + " Amount: " + payment.net_pay)

            SEND_CONFIRMATION(payment.employee_id)

            DISPLAY "Payment successful"

        ELSE

            CREATE_LOG("Payment failed for " + payment.employee_id)

            DISPLAY "Payment failed. Please try again."

        END IF

    END FOR

END FUNCTION
```

FR-08: PAYSLIP ACCESS FUNCTION:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: VIEW_MY_PAYSLIPS(employee_id)

Description: Allows employees to view and download their historical payslips.

FR: FR-08

```
FUNCTION VIEW_MY_PAYSLIPS(employee_id)

  IF user IS logged_in THEN

    WHILE true

      DISPLAY "1. View Payslips"

      DISPLAY "2. Download Payslip"

      DISPLAY "3. Exit"

      INPUT choice

      IF choice = 1 THEN

        payslips = GET_PAYSLIPS(user)

        DISPLAY payslips

      ELSE IF choice = 2 THEN

        INPUT payslip_id

        INPUT format (PDF/Excel)

        file = DOWNLOAD_PAYSLIP(user, payslip_id, format)

        IF file SUCCESS THEN

          DISPLAY "Download ready"

        ELSE

          DISPLAY "Error downloading"

        ENDIF

      ELSE IF choice = 3 THEN

        EXIT

      ENDIF

    ENDWHILE

  ENDIF

END
```

```

FUNCTION VIEW_MY_PAYSLIPS(employee_id) (cont)

// Functions

FUNCTION GET_PAYSLIPS(user)

    IF user.role = "employee" THEN

        RETURN GET_EMPLOYEE_PAYSLIPS(user.id)

    ELSE IF user.role = "hr" THEN

        RETURN GET_ALL_PAYSLIPS()

    ENDIF

ENDFUNCTION

FUNCTION DOWNLOAD_PAYSLIP(user, payslip_id, format)

    IF CAN_ACCESS_PAYSLIP(user, payslip_id) THEN

        payslip_data = GET_PAYSLIP_DATA(payslip_id)

        file = CREATE_FILE(payslip_data, format)

        LOG_DOWNLOAD(user, payslip_id)

        RETURN file

    ELSE

        RETURN "Access denied"

    ENDIF

ENDFUNCTION

FUNCTION CAN_ACCESS_PAYSLIP(user, payslip_id)

    IF user.role = "hr" THEN

        RETURN true

    ELSE IF user.role = "employee" THEN

        RETURN (payslip_id.employee_id = user.id)

    ELSE

        RETURN false

    ENDIF

END FUNCTION

```

FR-09: FINANCIAL REPORT GENERATION FUNCTION:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: GENERATE_FINANCIAL_REPORT(start_date, end_date) (and submodules)

Description: Allows finance team to generate financial reports.

FR: FR-09

```
FUNCTION GENERATE_FINANCIAL_REPORT(start_date, end_date)

    DISPLAY "Generating financial report from " + start_date + " to " +
end_date

// 1. Total salary expenditures
total_salary = QUERY_DATABASE("""
    SELECT SUM(net_pay) as total
    FROM payroll_db
    WHERE payment_date BETWEEN ? AND ?
""", start_date, end_date)

// 2. Total tax deductions
total_tax = QUERY_DATABASE("""
    SELECT SUM(deductions) as total
    FROM payroll_db
    WHERE payment_date BETWEEN ? AND ?
""", start_date, end_date)

// 3. Department-wise breakdown
department_costs = QUERY_DATABASE("""
    SELECT e.department, SUM(p.net_pay) as cost
    FROM payroll_db p
    JOIN employees_db e ON p.employee_id = e.id
    WHERE p.payment_date BETWEEN ? AND ?
    GROUP BY e.department
""", start_date, end_date)
```

```

(cont)

// 4. Display report

DISPLAY "=== FINANCIAL REPORT ==="

DISPLAY "Period: " + start_date + " to " + end_date

DISPLAY "Total Salary Paid: €" + total_salary

DISPLAY "Total Deductions: €" + total_tax

DISPLAY ""

DISPLAY "Department Breakdown:"

FOR EACH dept IN department_costs DO

    DISPLAY "    " + dept.department + ": €" + dept.cost

END FOR


DISPLAY "Export to PDF? (Y/N) "

INPUT export_choice

IF export_choice == "Y" THEN

    pdf_file = GENERATE_PDF_REPORT(total_salary, total_tax,
department_costs)

    INITIATE_DOWNLOAD(pdf_file)

END IF

END FUNCTION

```

FR-10: SALARY UPDATE MODULE:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: UPDATE_SALARY()

Description: Allows management to update the employee's base salary.

FR: FR-10

```
// Update Salary Function

FUNCTION UPDATE_SALARY(employee_id, new_salary)

    // Check if user has permission

    IF CURRENT_USER.role != "HR" AND CURRENT_USER.role != "Manager" THEN

        RETURN "Error: Access denied"

    ENDIF

    // Validate inputs

    IF employee_id IS EMPTY OR new_salary IS EMPTY THEN

        RETURN "Error: Missing data"

    ENDIF

    IF new_salary <= 0 THEN

        RETURN "Error: Invalid salary amount"

    ENDIF

    // Get employee record

    employee = FIND_EMPLOYEE(employee_id)

    IF employee NOT FOUND THEN

        RETURN "Error: Employee not found"

    ENDIF

    // Update salary in database

    old_salary = employee.salary

    UPDATE employee SET salary = new_salary WHERE id = employee_id

    // Log the change

    LOG_SALARY_CHANGE(employee_id, old_salary, new_salary, CURRENT_USER.id)

    RETURN "Salary updated successfully"

ENDFUNCTION
```

FR-11: USER AUTHENTICATION MODULE:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: **AUTHENTICATE_USER()**

Description: Nearly every task in PES requires user authentication. These functions are used whenever a user needs to be authenticated before they gain access to sensitive data.

FR: FR-11

```
FUNCTION LOGIN(username, password)

    // Check if user exists
    user = FIND_USER(username)

    IF user NOT FOUND THEN

        RETURN "Invalid username or password"

    ENDIF

    // Verify password
    IF password != user.password THEN

        RETURN "Invalid username or password"

    ENDIF

    // Return user info
    RETURN {

        user_id: user.id,

        username: user.username,

        role: user.role,

        is_logged_in: true

    }

ENDFUNCTION

FUNCTION CHECK_PERMISSION(user, required_role)

    IF user.role == required_role THEN

        RETURN true

    ELSE

        RETURN false

    ENDIF

ENDFUNCTION

FUNCTION LOGOUT(user)

    user.is_logged_in = false

    RETURN "Logged out successfully"

ENDFUNCTION
```

FR-12: PORTAL MODULES:

(Sketch omitted due to time constraints; however, the layout may be similarly composed from sketches already given.)

Function: PORTAL ()

Description: Various roles of PES will be routed to different portals. This is how the portals should be implemented.

FR: FR-12

```
FUNCTION EMPLOYEE_LOGIN(username, password)

    user = FIND_EMPLOYEE(username)

    IF user NOT FOUND OR password != user.password THEN

        RETURN "Login failed"

    ENDIF

    WHILE user.is_logged_in

        DISPLAY "Employee Portal:"

        DISPLAY "1. View my payslips"

        DISPLAY "2. Download payslip"

        DISPLAY "3. View my salary"

        DISPLAY "4. Logout"

        INPUT choice

        IF choice == 1 THEN

            payslips = GET_MY_PAYSLIPS(user.id)

            DISPLAY payslips

        ELSE IF choice == 2 THEN

            DOWNLOAD_MY_PAYSLIP(user.id)

        ELSE IF choice == 3 THEN

            DISPLAY "Salary: " + user.salary

        ELSE IF choice == 4 THEN

            user.is_logged_in = false

        ENDIF

    ENDWHILE

ENDFUNCTION
```

```

FUNCTION HR_LOGIN(username, password)

    hr_user = FIND_HR_USER(username)

    IF hr_user NOT FOUND OR password != hr_user.password THEN

        RETURN "Login failed"

    ENDIF

    WHILE hr_user.is_logged_in

        DISPLAY "HR Portal:"

        DISPLAY "1. View all payslips"

        DISPLAY "2. Update employee salary"

        DISPLAY "3. Generate reports"

        DISPLAY "4. Logout"

        INPUT choice

        IF choice == 1 THEN

            payslips = GET_ALL_PAYSLIPS()

            DISPLAY payslips

        ELSE IF choice == 2 THEN

            INPUT employee_id

            INPUT new_salary

            UPDATE_EMPLOYEE_SALARY(employee_id, new_salary)

        ELSE IF choice == 3 THEN

            GENERATE_HR_REPORTS()

        ELSE IF choice == 4 THEN

            hr_user.is_logged_in = false

        ENDIF

    ENDWHILE

ENDFUNCTION

```



```

FUNCTION MANAGER_LOGIN(username, password)

    manager = FIND_MANAGER(username)

    IF manager NOT FOUND OR password != manager.password THEN

        RETURN "Login failed"

    ENDIF

    WHILE manager.is_logged_in

        DISPLAY "Manager Portal:"

        DISPLAY "1. View team payslips"

        DISPLAY "2. Approve salary changes"

        DISPLAY "3. Team reports"

        DISPLAY "4. Logout"

        INPUT choice

        IF choice == 1 THEN

            team_payslips = GET_TEAM_PAYSLIPS(manager.team_id)

            DISPLAY team_payslips

        ELSE IF choice == 2 THEN

            APPROVE_SALARY_CHANGES()

        ELSE IF choice == 3 THEN

            GENERATE_TEAM_REPORTS(manager.team_id)

        ELSE IF choice == 4 THEN

            manager.is_logged_in = false

        ENDIF

    ENDWHILE

ENDFUNCTION

```

TEST CASES

Summary: In this section of the project progress report, you'll find the Test Cases created for PES. Multiple test cases may exist for a single functional requirement, but all functional requirements must at least have **one test case mapped to it**. The test cases are mapped to the functional requirements in the System Module Pseudocode section via their respective TC-ID and FR-ID identifiers.

TC-01: USER LOGIN AUTHENTICATION (FR-01, FR-11, FR-12)

Drafted by P. Maravgakis on 19/Nov/2025.

TC-ID: TC-01

FR-ID: FR-01, FR-11, FR-12

Test Case Type: Positive (*i.e. Not intended to cause bugs; using the login page as intended with correct data*)

Summary: This test case is for verifying that the login system:

1. Correctly authenticates users based on their credentials and role
2. Upon successful authentication, stores login data in local storage to stay logged in

Preconditions: User login data for Employee exists in the database.

Postconditions: User is logged in and routed to an appropriate portal.

Expected Results: The user receives a success message, then they are routed to “/Home”. The appropriate dashboard is displayed.

Steps	Test Data
1. Open the website. 2. Enter Test Data. 3. Click Login Button.	user data case sensitive Username: costasi Password: 77889 Role: employee

Actual Result: The success message is displayed. The user is correctly routed to “/Home”. The appropriate employee dashboard is displayed, meaning that the user data is locally stored to grant access rights.

Status: Success

TC-02: USER LOGIN WITH MISTYPED CORRECT DATA (FR-01, FR-11, FR-12)

Drafted by P. Maravgakis on 19/Nov/2025.

TC-ID: TC-02

FR-ID: FR-01, FR-11, FR-12

Test Case Type: Negative (*i.e. Intended to test control mechanisms; misusing the login page by using invalid data.*)

Summary: This test case is for verifying that the login system:

3. Correctly handles case-sensitive usernames and passwords by checking for case sensitivity
4. Correctly denies access for invalid attempts and doesn't route to "/Home" portal.

Preconditions: None.

Postconditions: User will have stayed in the login page.

Expected Results: The user receives an error message prompting them to re-enter login credentials.

Steps	Test Data
1. Open the website. 2. Enter Test Data. 3. Click Login Button.	user data case sensitive Username: CostasI Password: 77889 Role: employee

Actual Result: The error message is displayed, prompting the user to re-enter login credentials.

Status: Success

TC-03: USER LOGIN ATTEMPTED DESTRUCTIVE BYPASS (FR-01, FR-11, FR-12)

Drafted by P. Maravgakis on 19/Nov/2025.

TC-ID: TC-03

FR-ID: FR-01, FR-11, FR-12

Test Case Type: Destructive (*i.e. Aiming to discover what the system can handle before it breaks.*)

Summary: This test case is for verifying that the login system:

1. Can withstand malicious attempts to bypass the login system.
2. Can withstand load from a single user (specifically, spam-clicking buttons 100 times a second using autoclicker software.)

Preconditions: None.

Postconditions: User will have stayed in the login page.

Expected Results: The user is unable to enter “/Home” or access any of the dashboards and can only end up at the login page. The user also cannot break the login page or cause downtime to the server.

Steps	Test Data
1. Open the website. 2. Enter any data. 3. Click login button with autoclicker for 30s. 4. Attempt to change the url to “[websitename]/home”.	Username: any Password: any Role: any

Actual Result: The error message is displayed, prompting the user to re-enter login credentials if random credentials are entered. Spamclicking for 30s does not seem to cause significant server load. Upon trying to append “/Home” to the browser, the unauthenticated user is immediately rerouted back to the login page.

Status: Success

TC-04: USER LEAVE REQUEST FUNCTION WITH CORRECT INFORMATION (FR-04)

Done by Constantinos Stephanides

TC-ID: TC-04

Test Case Type: Positive (*i.e. Not intended to cause bugs; using the login page as intended with correct data*)

Summary: Testing whether the leave request function works properly:

1. Checks if the employee has more leave days than the requested amount
2. If verified, then the system registers their request in the database
3. The request is sent to the HR for approval
4. If verified, the status of the leave request is updated from “Pending” to “Accepted”
5. If not verified, the status of the leave request is updated from “Pending” to “Rejected”

Preconditions: Employee’s leave days are in the database

Postconditions: User is logged in and routed to an appropriate portal.

Expected Results: The user receives a success message and can check the status of the request from the “Leave Information” window

Steps	Test Data
1. Press on the “Leave Request” button. 2. Enter data requested for the leave. 3. Submit the form and wait for updates 4. Check status from the “Leave Information”	Leave type: Sick or Annual Starting Date Ending Date

Actual Result: Upon successful completion of the form, a message appears confirming the leave request while also storing it in the database.

TC-05: USER LEAVE REQUEST FUNCTION WITH INVALID INFORMATION (FR-04)

Done by Constantinos Stephanides

TC-ID: TC-05

Test Case Type: Negative

Summary: Testing whether the leave request function works properly:

1. If starting date is before the actual current date.
2. If starting date is after end date.
3. If duration is less than the number of leaves remaining.

Preconditions: Employee’s leave days are in the database

Postconditions: User is logged in and routed to an appropriate portal (Employee/HR/Team Manager Page)

Expected Results: The user receives a failure message and is asked to enter correct information

Steps	Test Data
-------	-----------

1. Press on the "Leave Request" button.
2. Enter data requested for the leave.
3. Submit the form and wait for updates

Leave type: Sick or Annual

Starting Date: after current date end date

Ending Date: before starting date

Actual Result: User can't select the dates before the current date (restricting by not allowing them to be selected). In addition, the end date isn't allowed to be the same as the start date.

TC-07A: EMPLOYEE LEAVE MANAGEMENT ACCEPT (FR-05, FR-04)

Drafted by P. Maravgakis on 25/Nov/2025.

TC-ID: TC-07A

FR-ID: FR-05, FR-04

Test Case Type: Positive

Summary: This test case is for verifying that the Employee Leave Management Submodule:

1. Is accessible by HR-class users, allowing them to access a list of pending leave requests.
2. Can accept a leave request pulled from the leaves database, and set its status from “pending” to “accepted”.

Preconditions: A leave request has been submitted (TC-04). The user must be logged into an HR-class account(TC-01).

Postconditions: A leave request will have been accepted.

Expected Results: The leave request will have been changed from “pending” to “accepted” within the leaves database, reflected on the webpage.

Steps	Test Data
1. Press the “Manage Employee Leaves” Button on the HR dashboard to bring up the leave management dialogue.	Username: tomasd
2. Let the dialog populate with entries from the leaves database whose status is set to “pending”.	Password: 12345
3. Click “Accept” on the selected leave request listing.	Role: HR

Actual Result: The logic implementation for this has yet to be fully implemented, but the frontend is.

Status: Pending

TC-07B: EMPLOYEE LEAVE MANAGEMENT REJECT (FR-05, FR-04)

Drafted by P. Maravgakis on 25/Nov/2025.

TC-ID: TC-07B

FR-ID: FR-05, FR-04

Test Case Type: Positive

Summary: This test case is for verifying that the Employee Leave Management Submodule:

1. Is accessible by HR-class users, allowing them to access a list of pending leave requests.
2. Can reject a leave request from the leaves database and change its status from “pending” to “rejected”.

Preconditions: A leave request has been submitted (TC-04). The user must be logged into an HR-class account(TC-01).

Postconditions: A leave request will have been rejected.

Expected Results: The leave request will have been changed from “pending” to “rejected” within the leaves database, reflected on the webpage.

Steps	Test Data
1. Press the “Manage Employee Leaves” Button on the HR dashboard to bring up the leave management dialog.	Username: tomasd
2. Let the dialog populate with entries from the leaves database whose status is set to “pending”.	Password: 12345
3. Click “Reject” on the selected leave request listing.	Role: HR

Actual Result: The logic implementation for this has yet to be fully implemented, but the frontend is.

Status: Pending

TC-08A: EMPLOYEE ONBOARDING ADD (FR-02)

Drafted by P. Maravgakis on 02/Dec/2025.

TC-ID: TC-08A

FR-ID: FR-02

Test Case Type: Positive

Summary: This test case is for verifying that the HR Employee Onboarding Management Submodule:

1. Is accessible by HR-class users, allowing them to access the employee onboarding submodule
2. Can “onboard” an employee; creating a new entry in the users table with their name, username, password, role saved.

Preconditions: The user must be logged into an HR-class account(TC-01). The user must also have new employee details on hand.

Postconditions: A new employee will have been onboarded.

Expected Results: A new employee will be entered into the users database, with a saved UID, Name, Username, Password and Role.

Steps	Test Data
1. While on the HR dashboard, click “Employee Onboard” on the HR actions module.	Name: JaneDoe
2. Viewing the employee list, click the “add new employee” button on it.	Username: jdoe
3. When prompted with “Name, Username, Password, Role” fields, enter the Test Data.	Password: 05120
4. Click ‘add new employee’. Wait for the database response.	Role: Employee

Actual Result: The logic implementation for this has yet to be fully implemented.

Status: Pending

TC-08B: EMPLOYEE ONBOARDING REMOVE (FR-02)

Drafted by P. Maravgakis on 02/Dec/2025.

TC-ID: TC-08B

FR-ID: FR-02

Test Case Type: Positive

Summary: This test case is for verifying that the HR Employee Onboarding Management Submodule:

1. Is accessible by HR-class users, allowing them to access the employee onboarding submodule
2. Can **remove** an employee from the employee list, deleting their database entry.

Preconditions: The user must be logged into an HR-class account(TC-01). The user must have an existing employee onboarded(TC-08).

Postconditions: An employee will have been removed from the employees list.

Expected Results: An employee from the existing employees list will have been selected, removed from the list, and the database will correctly delete their data.

Steps	Test Data
1. While on the HR dashboard, click "Employee Onboard" on the HR actions module.	Name: JaneDoe
2. Viewing the employee list, click the "edit existing employee" button on an employee listing.	Username: jdoe
3. Click the 'delete' button.	Password: 05120
4. Click confirm.	Role: Employee

Actual Result: The logic implementation for this has yet to be fully implemented.

Status: Pending

Drafted by P. Maravgakis on 03/Dec/2025.

TC-ID: TC-12E

FR-ID: FR-12, FR-01, FR-00A

Test Case Type: Positive

Summary: This test case is for verifying that the portal routing module:

1. Is capable of properly routing to the employee portal by logging into PES using an account with the role 'employee'.

Preconditions: An account with the role 'employee' exists.

Postconditions: The user will be logged into the system, authenticated, and routed to the employee portal.

Expected Results: The employee-role user will enter their credentials into the login screen, click 'log in', and will be routed to the employee portal.

Steps	Test Data
1. Enter the login screen, 2. Enter Test Data into user credentials. 3. Click 'sign in'	Name: costasi Password: 77889 Role: Employee

Actual Result: The employee role user is correctly routed into the employee portal once navigated to "/home".

Status: Success

TC-12H: HR PORTAL ACCESS (FR-12, FR-01, FR-00A)

Drafted by P. Maravgakis on 03/Dec/2025.

TC-ID: TC-12H

FR-ID: FR-12, FR-01, FR-00A

Test Case Type: Positive

Summary: This test case is for verifying that the portal routing module:

1. Is capable of properly routing to the HR portal by logging into PES using an account with the role 'HR'.

Preconditions: An account with the role 'HR' exists.

Postconditions: The user will be logged into the system, authenticated, and routed to the HR portal.

Expected Results: The HR-role user will enter their credentials into the login screen, click 'log in', and will be routed to the HR portal.

Steps	Test Data
1. Enter the login screen, 2. Enter Test Data into user credentials. 3. Click 'sign in'	Name: tomasd Password: 12345 Role: HR

Actual Result: The HR role user is correctly routed into the HR portal once navigated to "/home".

Status: Success

TC-12F: FINANCE PORTAL ACCESS (FR-12, FR-01, FR-00A)

Drafted by P. Maravgakis on 03/Dec/2025.

TC-ID: TC-12F

FR-ID: FR-12, FR-01, FR-00A

Test Case Type: Positive

Summary: This test case is for verifying that the portal routing module:

1. Is capable of properly routing to the Finance portal by logging into PES using an account with the role 'Finance'.

Preconditions: An account with the role 'Finance' exists.

Postconditions: The user will be logged into the system, authenticated, and routed to the Finance portal.

Expected Results: The Finance-role user will enter their credentials into the login screen, click 'log in', and will be routed to the Finance portal.

Steps	Test Data
1. Enter the login screen, 2. Enter Test Data into user credentials. 3. Click 'sign in'	Name: andreask Password: 99999 Role: Finance

Actual Result: The Finance role user is correctly routed into the Finance portal once navigated to "/home".

Status: Success

TC-12A: ADMIN PORTAL ACCESS (FR-12, FR-01, FR-00A)

Drafted by P. Maravgakis on 03/Dec/2025.

TC-ID: TC-12A

FR-ID: FR-12, FR-01, FR-00A

Test Case Type: Positive

Summary: This test case is for verifying that the portal routing module:

2. Is capable of properly routing to the Administrator portal by logging into PES using an account with the role 'Administrator'.

Preconditions: An account with the role 'Administrator' exists.

Postconditions: The user will be logged into the system, authenticated, and routed to the Administrator portal.

Expected Results: The Administrator-role user will enter their credentials into the login screen, click 'log in', and will be routed to the Administrator portal.

Steps	Test Data
1. Enter the login screen, 2. Enter Test Data into user credentials. 3. Click 'sign in'	Name: peterm Password: 00112 Role: Administrator

Actual Result: The Administrator role user is correctly routed into the Administrator portal once navigated to "/home".

Status: Success

TC-13: VIEWING AND DOWNLOADING PDF (FR-08)

Done by Andreas Karpasitis

TC-ID: TC-13

FR-ID: Fr-08

Test Case Type: Positive

Summary: This test case is for verifying that the view payslips button:

3. Is capable of properly displaying the PDF's that are generated from the data derived from the database.

Postconditions: The user will be logged into the system, authenticated, and routed to the respective portal.

Expected Results: The respective user will login into the system, move to the view my payslips and press the button

Steps	Test Data
4. Enter the login screen, 5. Enter Test Data into user credentials. 6. Click 'Sign in' 7. Click 'View my payslips'	Name: john Password: 123 Role: Employee

TC-14: EMPLOYEE ONBOARDING/OFFBOARDING (FR-02)

TC-ID: TC-14

FR-ID: Fr-02

Test Case Type: Positive

Summary: This test case is for verifying that the add remove employee is working as intended:

4. Is capable of adding or soft/hard removal of employees from the database.

Postconditions: The user will be logged into the system, authenticated, and routed to the respective portal (HR or Admin).

Expected Results: The respective user will login into the system, move to the view add/remove employees and press the button, select the user they want to remove and the user will be deleted from the database or if soft deletion they will be deactivated and not be able to login

Steps	Test Data
8. Enter the login screen,	Name: admin
9. Press add/remove employees	Password: admin123
10. Click 'Add employee'	Role: Admin
a. Enter Credentials of new employee	
b. New employee will now be available in DB	
11. Click on 'Actions'	Name: bob
12. Click on 'Deactivate'	Password: 123
a. User will not be able to login after deactivation	Role: HR
13. If admin - Click on delete permanently	
a. User will be completely removed from DB	

TC-15: UPDATE EMPLOYEE PERSONAL DATA(FR-02B)

Done by Andreas Karpasitis

TC-ID: TC-15

FR-ID: Fr-02b

Test Case Type: Positive

Summary: This test case is for verifying that the updating of the employee's data is working as intended:

5. Is capable of updating the personal data of the employee from the database.

Postconditions: The user will be logged into the system, authenticated, and routed to the respective portal (HR or Admin).

Expected Results: The respective user will login into the system, move to the view update employee details and press the button, select the user they want to update and the user will then be prompted to update the details of the selected user.

Steps	Test Data
14. Enter the login screen	Name: admin
15. Press 'Update employee details'	Password: admin123
16. Fill the form with the new details of the employee	Role: Admin
17. Submit the new form	
18. Details of employee in database will be updated	Name: bob
	Password: 123
	Role: HR

TC-16: PORTALS – EMPLOYEE PORTAL (FR-12)

Done by Constantinos Stephanides

TC-ID: TC-16

FR-ID: Fr-12

Test Case Type: None

Summary: Updated the employee portal to show the user: Current Salary, YTD Earnings and Leave Balance

Expected Results:

For user: “John”

Salary: 5400

Leave Balance: 12

id	username	salary	leave_balance
1	john	5400	12

Actual Results:

Employee Portal
ID: john

Login

Home

Employee Dashboard

\$

Current Salary

5,400

Monthly Gross Pay

Next Payment

Loading...

Estimated date

YTD Earnings

0

Year to date

Leave Balance

12 days

Available this year

TC-17: PORTALS – HR PORTAL (FR-12)

Done by Constantinos Stephanides

TC-ID: TC-17

FR-ID: Fr-12

Test Case Type: Positive

Summary: Updated the hr portal to show the user: Total Employees & Pending Approvals (Leaves)

Expected Results: Total Employees but 1 is inactive, therefore 8

For user: “tomasd” **Employees:** 8 **Pending Approvals:** 8

	id	username	is_active
	Fil...	Filter	Filter
1	1	john	1
2	2	jane	1
3	3	bob	1
4	4	admin	1
5	6	ewinis	0
6	7	tomasd	1
7	8	andreask	1
8	9	peterm	1
9	10	costasi	1

	leave_id	users_id	leave_type	start_date	end_date	status
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	Annual Leave	2024-04-01	2024-04-05	Approved
2	2	2	Sick Leave	2024-04-10	2024-04-11	Pending
3	3	3	Personal Leave	2024-04-15	2024-04-16	Pending
4	4	1	sick	2025-12-10	2025-12-13	Pending
5	5	1	annual	2025-12-10	2025-12-13	Pending
6	6	3	annual	2025-12-10	2025-12-14	Pending
7	7	6	annual	2025-12-19	2025-12-31	Pending
8	8	10	annual	2025-12-10	2025-12-11	Pending
9	9	10	annual	2025-12-10	2025-12-11	Pending

Results:

 **HR Management**
ID: tomasd

 Logout

Home

Welcome back, HR.



Total Employees

8

Total number of employees (Fetch total employees)



On Leave Today

12 (Mock)

Across all departments (Fetch all users on leave on current day)



Pending Approvals

8

Leave requests pending your approval (Fetch all pending requests)



Leave Balance

12 Days (Mock)

Available this year (Fetch current user's available leaves)

THIS PAGE INTENTIONALLY LEFT BLANK.

Drafted by P. Maravgakis on Dec 10, 2025

This manual aims to serve users of all four roles of possible user within the PES system. Whether you're an employee, human resources member, finance member or an administrator, you can find your respective dashboards described in the sections below. But first, every type of user needs to enter the login page.

THE LOGIN PAGE

Drafted by P. Maravgakis on Dec 10, 2025

Every user needs to authenticate themselves using the credentials given to them by the Human Resources (HR) department. When entering the PES, if you do not currently have an active session, you will be routed to the login page where you can enter your credentials to log in.

The image shows a login form for the 'Payment Enrollment System'. The form is white with a green border and contains the following elements: a green icon of a document with a checkmark, the title 'Payment Enrollment System', a subtitle 'Enter your credentials to access your account', a 'Role' dropdown menu with the text 'Select your role', an 'Employee ID' input field with the placeholder 'Enter your employee ID', a 'Password' input field with the placeholder 'Enter your password', and a green 'Sign In' button. To the right of the form, four green lines with circular endpoints point to the 'Role' dropdown, the 'Employee ID' field, the 'Password' field, and the 'Sign In' button. These lines are labeled with the numbers 1, 2, 3, and 4 respectively, corresponding to the list items on the right.

1. Role Selection
2. Employee ID
3. Password
4. Sign In

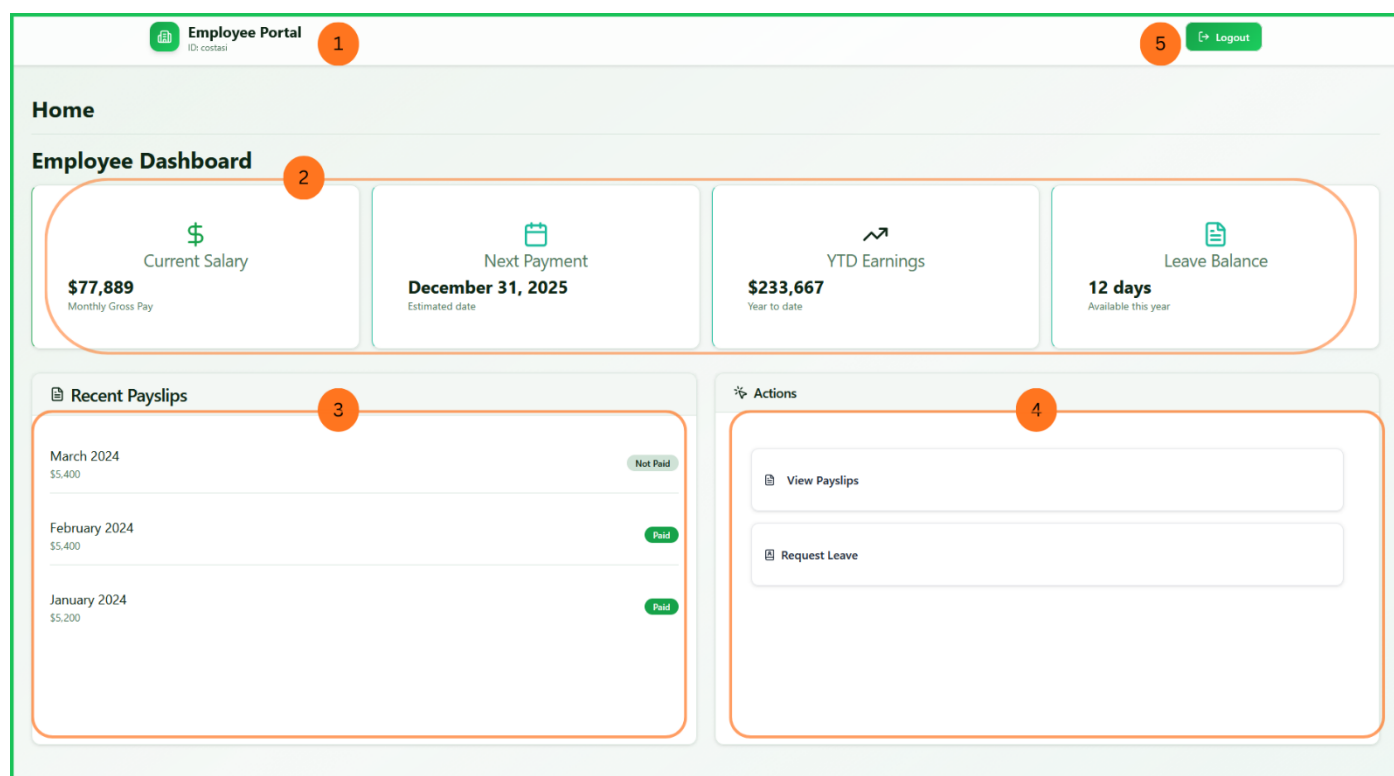
In the login screen (as shown above), you will be guided through the form as such:

1. **Role:** Enter your account role here. Your account role can be either Employee, HR, Finance or Administrator. For demonstration purposes, if one logs in with the wrong role selected, they will be warned about what their actual role is before being logged into their appropriate portal anyway; in a secure deployment of PES, however, entering the wrong role does not grant authentication.
2. **Employee ID:** Enter the username of your account within PES. This is a unique identifier that differs from your full name.
3. **Password:** Enter the password of your account within PES. The password is a string that is initially set by HR, but can be later changed by the user or administrator.
4. **Sign In:** Once you have completed the form, press this button to attempt to sign in.

An important note: The PES is software that is meant to be used internally within a company to disburse payrolls. As such, the management of accounts as well as the creation and deletion of accounts is handled internally by HR and Administrator-role users, as well as the IT technicians in charge of deploying the PES for a specific company.

THE EMPLOYEE PORTAL

Drafted by P. Maravgakis on Dec 10, 2025

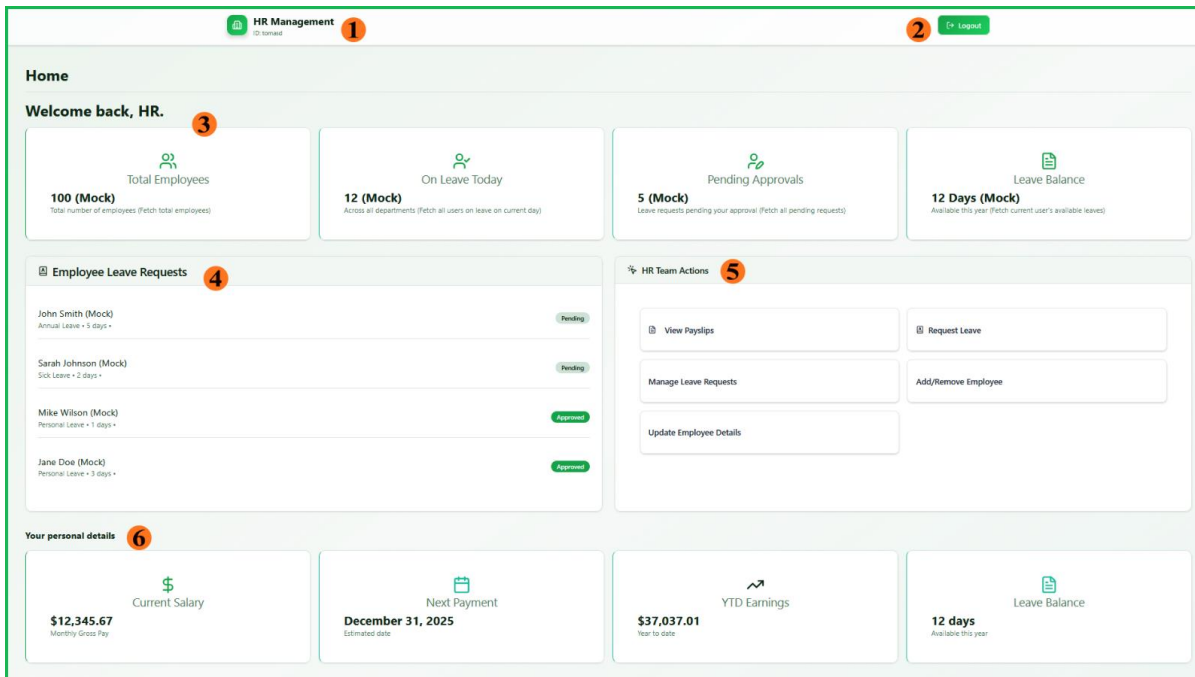


As an Employee, your dashboard is comprised of the following modules:

1. **Header:** The header tells you where you are on the webpage, helping the user get situated with the workspace. It provides the user's ID below the portal specification, as well as housing the Logout module.
2. **Employee Dashboard:** These cards allow you to get a quick glimpse at your most important statistics with regards to your payroll: salary, your upcoming payment, year-to-date earnings, as well as your current leave balance measured in days available in the current year.
3. **Recent Payslips:** A comprehensive list of your recent payslips. This card is scrollable and can show you payslips listed from the most recent one to the oldest one.
4. **Employee Actions:** Allows the user to perform various actions related to their role:
 - a. **View Payslips:** Views the user's payslips issued at every payment cycle, allowing the user to view their comprehensive breakdowns as well as their status (whether they are pending or have been disbursed to them yet).
 - b. **Request Leave:** Opens a calendar dialog that allows the user to request a leave of absence. This sends a pending leave request to HR and Administration, and, once approved, deducts from the user's leave balance and places the leave on their account record.
5. **Logout:** Logs the user out of their portal, sending them back to The Login Page.

THE HR PORTAL

Drafted by P. Maravgakis on Dec 10, 2025



As an Employee, your dashboard is comprised of the following modules:

1. **Header:** The header tells you where you are on the webpage, helping the user get situated with the workspace. It provides the user's ID below the portal specification, as well as housing the Logout module.
2. **Logout:** Logs the user out of their portal, sending them back to The Login Page.
3. **HR Dashboard:** These cards allow you to get a quick glimpse at an HR member's important statistics: the total employees currently active, the employees on leave today, as well as pending leave approvals, et cetera.
4. **Employee Leave Requests:** A comprehensive list of an HR member's leave requests. This card is scrollable and can show you leave requests listed from the most recent one to the oldest one.
5. **HR Actions:** Allows the user to perform various actions related to their role:
 - a. **View Payslips:** Views the user's payslips issued at every payment cycle, allowing the user to view their comprehensive breakdowns as well as their status (whether they are pending or have been disbursed to them yet).
 - b. **Request Leave:** Opens a calendar dialog that allows the user to request a leave of absence for oneself. This sends a pending leave request to HR and Administration, and, once approved, deducts from the user's leave balance and places the leave on their account record.
 - c. **Manage Leave Requests:** Allows an HR user to manage the pending leave requests within PES.
 - d. **Add/Remove Employee:** Allows an HR user to either add or remove a user, or alternatively temporarily deactivate an account.
 - e. **Update Employee Details:** Allows an HR user to edit the details of a user, such as name, leave balance, et cetera.
6. **Employee Dashboard:** These cards allow you to get a quick glimpse at your most important statistics with regards to your payroll: salary, your upcoming payment, year-to-date earnings, as well as your current leave balance measured in days available in the current year.

Finance Dashboard
ID: andreaks

Logout

Home

Welcome back, (Name).

Payroll Report
\$750,000 (Mock)
Total payroll (Fetch from most recent finance log, which'll have a summation of all user salaries; don't calculate it here that's too costly im SERIOUS)

Processed Payments
70 (Mock)
Processed this month (Fetch from payslips marked as paid in the current month ig)

Pending Payments
30 (Mock)
Pending salary payments (Fetch from payslips set as pending)

Budget Utilization
75% (Mock)
YTD vs Budget (Divide fetched payroll report by annual budget from finance logs idk im not an economist also this might be calcd and saved in payroll db)

Recent Payroll Reports

Pending Disbursements

Recent Transactions

March 2024
\$1,248,000 • 248 employees • Mar 31
Processing

February 2024
\$1,236,000 • 246 employees • Feb 29
Completed

January 2024
\$1,224,000 • 244 employees • Jan 31
Completed

Costantinos Coleslaw Employee
Due: Mar 31 **\$5,400**
Net pay

Andreas Ambidextrous HR
Due: Mar 31 **\$6,200**
Net pay

Tommy Toblerone Finance
Due: Mar 31 **\$4,800**
Net pay

Salary Payment
Mar 25 **\$5,400**

Bonus Payment
Mar 20 **\$1,200**

Reimbursement
Mar 18 **\$300**

Personal Payslips

Finance Team Actions

March 2024
\$9,999 Not Paid

February 2024
\$8,204 Paid

January 2024
\$3,030 Paid

View Payslips

Request Leave

Manage Payroll Report

Manage Payments

Your personal details

Current Salary
\$99,999.99
Monthly Gross Pay

Next Payment
December 31, 2025
Estimated date

YTD Earnings
\$299,999.97
Year to date

Leave Balance
12 days
Available this year

As an Employee, your dashboard is comprised of the following modules:

- Header:** The header tells you where you are on the webpage, helping the user get situated with the workspace. It provides the user's ID below the portal specification, as well as housing the Logout module.
- Logout:** Logs the user out of their portal, sending them back to The Login Page.
- Finance Dashboard:** These cards allow you to get a quick glimpse at a Finance Team member's important statistics: the payroll report, payments processed this month, as well as pending payments, et cetera.
- Recent Payroll Reports:** These reports concern the company's monthly reports in terms of payroll disbursements in total throughout the company.
- Pending Disbursements:** Allows the observation of currently pending payroll disbursements for employees.
- Recent Transactions:** Allows the observation of recently completed transactions. These are pulled from processed payment disbursements out to employees.
- Finance Actions:** Allows the user to perform various actions related to their role:

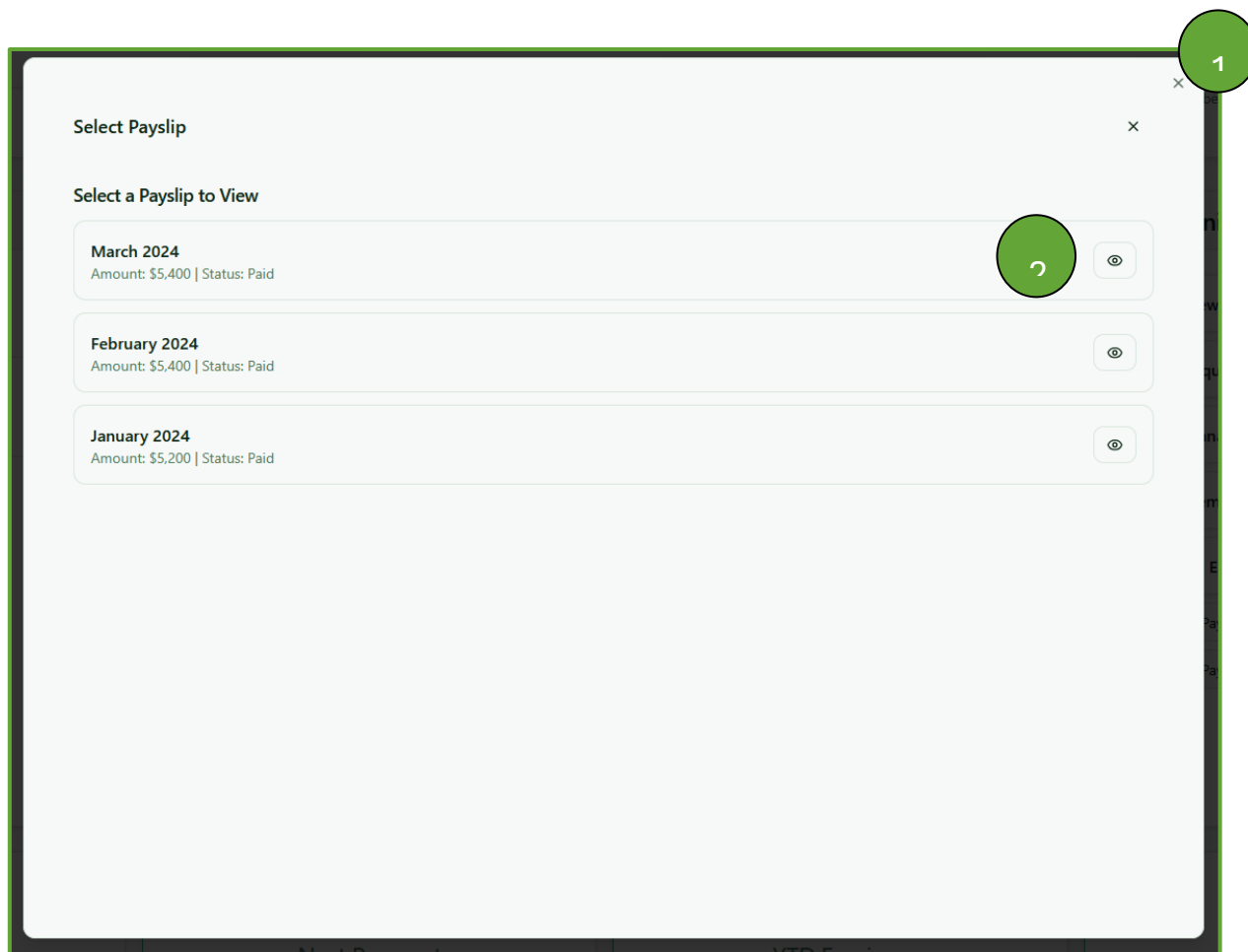
- a. **View Payslips:** Views the user's payslips issued at every payment cycle, allowing the user to view their comprehensive breakdowns as well as their status (whether they are pending or have been disbursed to them yet).
 - b. **Request Leave:** Opens a calendar dialog that allows the user to request a leave of absence for oneself. This sends a pending leave request to HR and Administration, and, once approved, deducts from the user's leave balance and places the leave on their account record.
 - c. **Manage Payroll Report:** Opens a dialog that allows the Finance-class user to calculate the payroll report for the whole company. This includes the payroll disbursements as well as any liabilities or deductibles.
 - d. **Manage Payments:** Opens a dialog that allows the Finance-class user to calculate the payroll disbursement for individual employees. Inside the dialog, the Finance-class user can observe the currently pending payrolls, as well as access more details about them. If the Finance-class user is satisfied with the payroll report for the individual employee, they can choose to accept the payroll report and run the disbursement, marking the particular pending payment as processed.
8. **Employee Dashboard:** These cards allow you to get a quick glimpse at your most important statistics with regards to your payroll: salary, your upcoming payment, year-to-date earnings, as well as your current leave balance measured in days available in the current year.

SUBMODULES

Drafted by Peter Maravgakis.

In this section you can observe the various different submodules available to members of PES. These submodules are granted to the four different kinds of accounts, Employee, HR, Finance and Administrator, based on a least-privilege principle, and once access is granted to these submodules they operate in a feature-complete manner.

VIEW PAYSLIPS



1. **Close Buttons:** The 'X' button on the outside closes the "View Payslip" submodule. The one on the inside can be used while viewing a payslip(see 2) to close the payslip and view a different one.
2. **Payslip List:** Initially allows the observation of the payslip's title, concerning the month that it was issued, as well as quickly accesible information about the total amount(net pay) and the status of the payroll regarding it.

The screenshot shows a 'Request Leave' form within a 'Personal Leave Calendar' context. The form has a title bar with a close button (X) labeled 1. Below the title is a subtitle: 'Submit a new leave request. Your current leave balance will be checked.' A section labeled 'Your Leave Balance' (2) shows '12 days available'. The 'Leave Type' (3) is a dropdown menu with 'Select leave type'. The 'Start Date' (4) and 'End Date' (5) are date pickers with 'mm / dd / yyyy' format. The 'Reason (Optional)' (6) is a text area with the placeholder 'Enter reason for leave (optional)'. At the bottom are 'Cancel' and 'Submit Request' buttons, with the 'Submit Request' button labeled 7.

1. **Close Button:** Clicking the 'X' exits the Request Personal Leave submodule.
2. **Leave Balance:** Shows you the leave balance of the currently logged in account. This number is detracted from when a leave request is approved.
3. **Leave Type:** Select the type of leave that you want to submit. Currently, the implementation allows you to select "Sick" and "Annual" leave.
4. **Start Date:** Select the start date of the leave request. This date needs to be a date in the future.
5. **End date:** Select the end date of the leave request. This date needs to be a date after the start date.
6. **Reason:** Set a reason for the leave request. This is a note for the leave requester to set, and is optional.
7. **Cancel/Submit:** If satisfied with the leave request, clicking submit will submit the request and set it to "pending". If unsatisfied, clicking cancel will discard the leave request and close the submodule.

MANAGE LEAVE REQUESTS

The screenshot shows the 'Manage Leave Requests' interface. At the top, there is a title bar with a close button (X) and an 'Export CSV' button. Below this is a search bar and two dropdown filters for 'All Status' and 'All Types', along with a 'Refresh' button. The main area contains a table of leave requests. The table has columns for Employee, Leave Type, Date Range, Days, Status, Reason, and Actions. A context menu is open over the 'Actions' column of the second row, showing options: 'View Details', 'Approve Leave', and 'Reject Leave'. At the bottom, there is a 'Showing 21 of 21 requests' indicator and a 'Close' button.

Employee	Leave Type	Date Range	Days	Status	Reason	Actions
Bob Johnson HR • EMP003	Annual Leave	Dec 21, 2025 - Dec 22, 2025 Submitted: Dec 19, 2025	2 days	Pending	No reason provided	⋮
Bob Johnson HR • EMP003	Annual Leave	Dec 20, 2025 - Dec 31, 2025 Submitted: Dec 19, 2025	12 days	Approved	No reason provided	⋮
Bob Johnson HR • EMP003	Sick Leave	Dec 20, 2025 - Dec 31, 2025 Submitted: Dec 19, 2025	12 days	Approved	No reason provided	⋮
Bob Johnson HR • EMP003	Sick Leave	Dec 20, 2025 - Dec 31, 2025 Submitted: Dec 19, 2025	12 days	Approved	No reason provided	⋮
Bob Johnson HR • EMP003	Annual Leave	Dec 20, 2025 - Dec 31, 2025 Submitted: Dec 19, 2025	12 days	Approved	No reason provided	⋮

1. **Close Button:** Clicking the 'X' exits the submodule.
2. **Export CSV:** Exports all leaves in the database into a CSV file.
3. **Filters:** Allows for filtering of the entries in 4 based on the status of the leave request as well as its type.
4. **Leave Request List:** A list of leave requests made by users of PES. Details can be
5. **Actions:** Opens a context menu that allows the user to choose an action for the selected leave request. You can view details about it, approve it, or reject it. Doing so changes its status, and is reflected within the database.
6. **Close:** Exit the submodule.

EMPLOYEE MANAGEMENT

The screenshot displays the 'Employee Management' interface. On the left, an 'Add Employee' dialog box is open, featuring input fields for Username, Password, Name, Position (a dropdown menu currently set to 'Employee'), Employee ID, and Salary. It includes 'Cancel' and 'Add Employee' buttons. The main area shows a table of employees with columns for Name, Position, Department, Status, and Actions. A search bar and a 'Refresh' button are at the top of the table. A 'Deactivate' button is visible in the Actions column for the first employee, 'Admin User'.

Name	Position	Department	Status	Actions
Admin User @admin	Administrator	Administration	Active	Deactivate
Andreas Karpasitis @andreask	Finance	-	Active	
Bob Johnson @bob	HR	Human Resources	Active	
Costantine Stephanides @costasi	Employee	-	Active	
Jane Doe @jane	Manager	Management	Active	
John Smith @john	Employee	Engineering	Active	
Peter Maravgakis @peterm	Administrator	-	Active	
Tomas Demosthenous @tomasd	HR	-	Active	
andreas @ewinis	Manager	Sales	Inactive	

1. **Close Button:** Clicking the 'X' exits the submodule.
2. **Search:** Filters by employee name.
3. **Refresh:** Refresh the employee list, fetching the most recent list of employees from the database.
4. **Add Employee:** Opens the dialogue box for adding a new employee(Check 7!)
5. **Employee List:** The list of users of PES, featuring the name, ID, position, department, status and action options for each entry.
6. **Actions:** Allows you to deactivate other user accounts, which halts their authorization to enter PES. This means the deactivated user won't be able to log in, or access submodules.
7. **Add Employee:** The form for adding a new employee. It prompts you to enter all key details of the new user: username, password, name, position, employeeid, salary.
8. **Submit/Cancel:** If satisfied with the new user entry, clicking submit will create a new user account. If unsatisfied, clicking cancel will discard the leave request and close the submodule.

UPDATE EMPLOYEE DETAILS

The screenshot shows a web application interface for updating employee details. It features a title bar with a close button (1), a search bar (2), and a refresh button. Below is a table of employees with columns for Name, Position, Department, Salary, Status, and Actions. A context menu (4) is open for the 'Admin User' row, showing an 'Update Details' option. Numbered callouts highlight these key UI elements.

Name	Position	Department	Salary	Status	Actions
Admin User @admin	Administrator	Administration	\$10,000	Active	⋮ Update Details
Andreas Karpasitis @andreask	Finance	-	\$99,999.99	Active	⋮
Bob Johnson @bob	HR	Human Resources	\$4,700	Active	⋮
Costantine Stephanides @costasi	Employee	-	\$77,889	Active	⋮
Jane Doe @jane	Manager	Management	\$7,200	Active	⋮
John Smith @john	Employee	Engineering	\$5,400	Active	⋮
Peter Maravgakis @peterm	Administrator	-	\$11,223.33	Active	⋮
Tomas Demosthenous @tomasd	HR	-	\$12,345.67	Active	⋮
andreas	Manager	Sales	\$6,500	Inactive	⋮

1. **Close Button:** Clicking the 'X' exits the submodule.
2. **Filter:** Allows the user to filter through employees by name. Clicking refresh refreshes the list.
3. **Employee List:** Allows the user to examine the list of currently active employees, as well as their traits from name, ID, position, department, salary, status to actions that may be taken for them.
4. **Actions:** Shows context actions for the user listing; the only option is to update the details of the selected user, which will prompt the user to change the details of the user's account.